



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



PLATAFORMA WEB INTEGRAL PER A LA CONFIGURACIÓ, MANTENIMENT I VISUALITZACIÓ DE SISTEMES DE VIDEOVIGILÀNCIA IP

JAN ROSELL CABA

Director/a

FERNANDO GALLEGU VILALTELLA (LANACCESS TELECOM, SA)

Ponent: CARLES FARRE TOST (Departament d'Enginyeria de Serveis i Sistemes d'Informació)

Titulació

Grau en Enginyeria Informàtica (Enginyeria del Software)

Memòria del treball de fi de grau

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Resum

Aquest projecte presenta el disseny i desenvolupament de **Web Core View**, una plataforma web moderna destinada a la gestió de videogravadors de seguretat. El treball neix d'una doble necessitat de Lanaccess Telecom, SA: d'una banda, substituir la seva plataforma web de gestió *legacy*, construïda originalment als anys 90 amb tecnologies avui obsoletes com els controls **ActiveX**, per una solució moderna que garanteixi ubiqüitat, seguretat i sostenibilitat; de l'altra, disposar d'una versió autònoma i simplificada orientada a empreses petites que gestionen un únic videogravador, sense necessitat de dependre d'una aplicació centralitzada connectada a tot el parc de dispositius. La solució s'ha implementat com una interfície web d'administració embedded utilitzant **Angular 19** i el nou paradigma de reactivitat basat en **Signals**. Els dos components tecnològics més rellevants del projecte són el motor de configuració dinàmic (*Schema-driven*), que permet desacoblar la interfície de l'evolució del *firmware*, i la integració del protocol **WebRTC** per a la visualització de vídeo de baixa latència (< 300 ms) sense necessitat de *plugins* externs. Els resultats validen que la solució web iguala o millora el rendiment de les eines clàssiques.

Abstract

This project presents the design and development of **Web Core View**, a modern web platform for the management of security video recorders. The project arises from a twofold need at Lanaccess Telecom, SA: on one hand, to replace its legacy web management platform, originally built in the late 1990s using obsolete technologies such as **ActiveX** controls, with a modern solution that ensures ubiquity, security and sustainability; on the other hand, to provide a standalone, lightweight version aimed at small businesses that manage a single video recorder, without requiring a centralized application connected to the entire device fleet. The solution has been implemented as an embedded web administration interface using **Angular 19** and the new reactivity paradigm based on **Signals**. The two most significant technological pillars of the project are the dynamic configuration engine (*Schema-driven*), which allows decoupling the interface from firmware evolution, and the integration of the **WebRTC** protocol for low-latency video streaming ($< 300\text{ms}$) without the need for external plugins. The results validate that the web solution matches or improves the performance of the company's legacy tools.

Índex

1	Introducció i Context	1
1.1	Context del projecte	1
1.2	Identificació del problema i motivació	1
1.3	Estat de l'art i alternatives existents	2
1.3.1	Estat de l'art	2
1.3.2	Taula comparativa	3
1.4	Anàlisi d'alternatives i justificació de la solució	3
1.4.1	Alternativa de Framework Frontend: Angular 19 vs. React / Vue	3
1.4.2	Alternativa de vídeo: Arquitectura Híbrida WebRTC i HLS vs. RTSP / RTMP	4
1.4.3	Alternativa de Base de Dades per a Alarmes: SQLite3 vs. PostgreSQL / MongoDB	4
1.4.4	Alternativa de Còdec de Vídeo: H.264 i H.265 coexistents segons Maquinari i Càmera	5
1.4.5	Alternativa de configuració: Estàtica vs. Schema-driven	5
1.4.6	Alternativa de gestió d'estat: Redux vs. Angular Signals	6
1.5	Solució proposada i valor afegit	6
1.6	Objectius del projecte	7
1.6.1	Objectiu general	7
1.6.2	Objectius específics	7
1.7	Abast i Obstacles	7
1.7.1	Abast funcional	7
1.7.2	Obstacles detectats	7
1.8	Actors implicats (Stakeholders)	8
2	Gestió del projecte	9
2.1	Metodologia i eines de treball	9
2.2	Planificació temporal	10
2.2.1	Descripció de les fases i tasques	10
2.2.2	Recursos identificats	10
2.2.3	Estimació d'hores i Diagrama de Gantt	11
2.2.4	Gestió del risc i desviacions reals	12
2.2.5	Desviacions i impacte econòmic	14
2.2.6	Estat actual del projecte	14
2.3	Anàlisi econòmica	14
2.3.1	Costos de personal	15

2.3.2	Costos totals i resum econòmic	15
2.3.3	Control de gestió i seguiment de desviacions	15
2.4	Tancament del projecte i desviacions finals	16
2.4.1	Tancament temporal: Planificació prevista vs. real	16
2.4.2	Tancament econòmic: Pressupost estimat vs. final	17
2.5	Anàlisi de sostenibilitat i implicacions ètiques	18
3	Especificació de Requisits	20
3.1	Metodologia d'extracció de requisits	20
3.2	Identificació d'actors	20
3.3	Històries d'Usuari (User Stories)	21
3.3.1	Mòdul de Visualització i Reproducció de Vídeo	21
3.3.2	Mòdul de Gestió d'Alarmes	21
3.3.3	Mòdul de Configuració Dinàmica i Xarxa	22
3.3.4	Mòdul de Manteniment	22
3.3.5	Flux d'Interacció (User Journey)	22
3.4	Requisits Funcionals (RF)	22
3.4.1	RF-1: Gestió de l'Accés i Seguretat	23
3.4.2	RF-2: Visualització Multimèdia	23
3.4.3	RF-3: Motor de Configuració Dinàmica	24
3.5	Requisits No Funcionals (RNF)	24
3.5.1	RNF-1: Rendiment i Eficiència	25
3.5.2	RNF-2: Robustesa i Disponibilitat	25
3.5.3	RNF-3: Compatibilitat i Portabilitat	25
3.6	Casos d'Ús detallats	25
3.6.1	UC-01: Gestió de l'actualització de firmware	26
3.6.2	UC-02: Consulta de gravacions mitjançant Timeline	26
3.7	Restriccions del sistema	26
3.8	Marc normatiu i regulacions	27
3.8.1	Protecció de Dades de Caràcter Personal (RGPD i LOPDGDD)	27
3.8.2	Seguretat en la comunicació i Xifratge	27
3.8.3	Llei de Seguretat Privada i Norma UNE-EN 62676	28
4	Arquitectura i Disseny	29
4.1	Arquitectura Global del Sistema	29
4.1.1	Capa de comunicació i Microserveis	30
4.1.2	Seguretat Defensiva contra vectors d'atac OWASP	30
4.1.3	Estratègia de Fallback i robustesa del canal de vídeo	31
4.2	Disseny del Frontend (Angular 19)	32
4.2.1	Arquitectura Modular i Standalone Components	32
4.2.2	Gestió de l'Estat amb Angular Signals	32
4.3	El Motor Schema-Driven	33
4.3.1	Lògica de funcionament	33
4.4	Disseny de la Persistència i dades	34
4.4.1	Model de dades de l'aplicació	34

5	Implementació	36
5.1	Entorn de desenvolupament i tecnologies	36
5.2	Mòdul d'Autenticació i Core	36
5.2.1	Protecció de rutes i Functional Guards	37
5.3	Implementació del Motor Schema-Driven	38
5.3.1	El component genèric de configuració	38
5.3.2	Optimització del payload mitjançant deltes de canvis	39
5.3.3	Optimització de Cicle de Vida i Fil d'Execució	41
5.4	Visualització de Vídeo Real-Time (WebRTC)	42
5.4.1	Negociació SDP i intercanvi de candidats	42
5.4.2	Orquestració Reactiva amb Signals i Effects	45
5.5	Sistema d'Alarmes i Comunicació asíncrona	46
5.5.1	Estratègia de reconexió amb Backoff exponencial	46
5.6	Mòdul de Manteniment i Logs	48
5.7	Reproducció de Gravacions i Timeline Històrica	48
6	Validació i Proves	50
6.1	Estratègia de proves	50
6.2	Proves Unitàries i de Lògica	50
6.2.1	Validació del Parser de metadades	51
6.3	Resultats de rendiment i latència	51
6.3.1	Consum de CPU, Memòria i Absència de Fugues (Memory Leaks)	52
6.4	Proves de Robustesa i Comunicació asíncrona	52
6.4.1	Escenari de tall de connectivitat	52
6.5	Validació d'Acceptació (UAT)	53
7	Integració de Coneixements	54
7.1	Especialitat en Enginyeria del Software	54
7.1.1	Projecte d'Enginyeria del Software (PES)	54
7.1.2	Gestió de Projectes de Software (GPS)	54
7.1.3	Enginyeria de Requisits (ER)	55
7.1.4	Disseny de Bases de Dades (DBD)	55
7.1.5	Arquitectura del Software (AS)	55
7.2	Bases de l'Enginyeria del Software	56
7.2.1	Introducció a l'Enginyeria del Software (IES)	56
7.3	Core d'Enginyeria Informàtica (Troncals i Comuns)	56
7.3.1	Projecte de Programació (PROP)	56
7.3.2	Bases de Dades (BD)	56
7.3.3	Xarxes de Computadors (XC)	56
7.3.4	Sistemes Operatius (SO)	57
7.4	Interacció i Disseny d'Interfícies (IDI)	57
7.5	Assoliment de les competències de l'especialitat	57
7.5.1	Competències assolides en profunditat	58
7.5.2	Competències assolides en un nivell notable (Bastant)	58
7.5.3	Competències assolides en un nivell introductori (Una mica)	59
7.6	Aprenentatge Autònom	60

8	Conclusions i Treball Futur	61
8.1	Assoliment dels objectius	61
8.2	Treball futur	62
8.3	Conclusions personals	62
	Bibliografia	63
A	Annexos	65

Índex de taules

1.1	Comparativa de solucions de mercat.	3
2.1	Estimació d'hores i recursos utilitzats a cada tasca.	12
2.2	Matriu de gestió de riscos i plans de contingència.	13
2.3	Costos de recursos humans per rol.	15
2.4	Comparativa de la planificació temporal prevista enfront del temps real d'execució.	17
2.5	Comparativa del cost de recursos humans estimat enfront de l'executat real.	18
3.1	Especificació detallada del Cas d'Ús UC-01.	26
3.2	Especificació detallada del Cas d'Ús UC-02.	27
6.1	Comparativa de latència segons protocol de comunicació.	51

Índex de figures

2.1	Mapa mental del stack tecnològic i d'enginyeria aplicat al projecte. . .	10
2.2	Diagrama de Gantt del projecte.	12
3.1	Diagrama de flux d'interacció de l'usuari (User Journey) en la resolució d'una incidència de seguretat.	23
3.2	Diagrama UML de Casos d'Ús.	25
4.1	Diagrama de desplegament físic i lògic del sistema WebCoreView. . .	29
4.2	Diagrama de components de l'arquitectura de l'aplicació Angular i la seva integració amb el backend.	32
4.3	Diagrama de mòduls de l'aplicació Angular i relació amb els components compartits i serveis de la capa Core.	33
4.4	Diagrama de classes de les entitats principals del sistema WebCoreView.	35
5.1	Diagrama de seqüència del flux de control d'inici de sessió.	37
5.2	Captura de la pantalla d'inici de sessió (Login) de l'aplicació.	38
5.3	Captura de pantalla d'un formulari de configuració avançat generat dinàmicament pel motor Schema-Driven (amb <i>toggles</i> , <i>sliders</i> i <i>selects</i>). . .	39
5.4	Diagrama de seqüència del procés d'adquisició i alliberament de Lock de configuració.	40
5.5	Diagrama d'activitat de la lògica de connexió i reconexió de vídeo. .	43
5.6	Diagrama de seqüència de l'establiment de streaming de vídeo.	44
5.7	Vista principal de l'operador mostrant el mosaic de visualització de vídeo en directe.	44
5.8	Diagrama d'estats del cicle de vida de les alarmes.	46
5.9	Captura del panell mostrant una alerta o notificació d'alarma en temps real mitjançant WebSockets.	47
5.10	Captura del visor de registres (Logs).	48
5.11	Captura de la barra de progrés d'actualització de firmware.	49
5.12	Vista del reproductor de vídeo històric amb el component interactiu de la línia de temps (<i>Timeline</i>).	49

Capítol 1

Introducció i Context

En aquest primer capítol es defineix el marc en el qual es desenvolupa el projecte, s'analitza la problemàtica de la plataforma web *legacy* de l'empresa Lanaccess i es justifica la necessitat de substituir-la per una solució moderna. Així mateix, es detallen els objectius, l'abast i els actors implicats en el cicle de vida del software.

1.1 Context del projecte

El present Treball de Final de Grau (TFG) consisteix en el disseny i desenvolupament de **Web Core View**, una aplicació web moderna destinada a la gestió centralitzada dels videogravadors de l'empresa Lanaccess Telecom, SA. El projecte es realitza en col·laboració amb aquesta empresa tecnològica, especialitzada en el desenvolupament de solucions professionals de videovigilància i seguretat electrònica.

El treball s'emmarca dins de la **Modalitat B** del TFG i dins de l'especialitat d'Enginyeria de Software, posant èmfasi en arquitectures frontend modernes, seguretat en la comunicació i integració de protocols de *streaming* en temps real. L'objectiu principal és liderar la modernització tecnològica de l'empresa, substituint l'aplicació web *legacy* de gestió — construïda als anys 90 amb tecnologies avui obsoletes com **ActiveX** — per una interfície web de control i administració embedded basada en l'ecosistema Angular amb arquitectura i protocols moderns.

1.2 Identificació del problema i motivació

Històricament, la gestió operativa i tècnica dels gravadors de vídeo de Lanaccess s'ha realitzat mitjançant una aplicació web desenvolupada als anys 90 que depenia de tecnologies avui completament obsoletes, com els controls **ActiveX** de Microsoft, entre d'altres. Aquesta plataforma *legacy* presenta una obsolescència tecnològica severa que es tradueix en els següents problemes:

- **Dependència de tecnologies obsoletes:** L'ús de controls ActiveX restringeix el funcionament de l'aplicació a navegadors antics (Internet Explorer), incompatible amb els navegadors web moderns (Chrome, Firefox, Edge), que han eliminat completament el suport per a *plugins* propietaris per raons de seguretat.

- **Rigidesa davant hardware heterogeni:** Els gravadors moderns posseeixen capacitats i *firmwares* molt diversos. La interfície estàtica de la plataforma *legacy* no pot adaptar-se eficientment a la disparitat de paràmetres (IP, ports, lògiques d'alarma) de cada equip.
- **Obsolescència d'experiència d'usuari (UX):** La interfície actual, dissenyada fa més de dues dècades, requereix una corba d'aprenentatge elevada i no compleix els estàndards de reactivitat i claredat que exigeixen els operadors de seguretat actuals.

Paral·lelament a l'obsolescència de la plataforma existent, Lanaccess ha identificat una **necessitat de mercat addicional**: oferir una versió simplificada i autònoma del programari de gestió, orientada a **empreses petites** que disposen d'un únic videogravador. El model anterior estava concebut exclusivament per a grans instal·lacions centralitzades, on una única aplicació es connecta simultàniament a tot el parc de dispositius. Aquesta arquitectura resulta sobredimensionada i innecessàriament complexa per a clients amb una sola unitat de gravació. Així, el projecte *Web Core View* neix amb una doble finalitat: modernitzar la plataforma tecnològica obsoleta i, alhora, proporcionar una solució *embedded* lleugera que s'executi directament en el propi gravador, permetent la gestió autònoma de l'equip des de qualsevol navegador web sense requerir infraestructura addicional.

1.3 Estat de l'art i alternatives existents

1.3.1 Estat de l'art

Actualment, el mercat de la videovigilància IP està dominat per grans fabricants com Hikvision o Dahua, que ofereixen interfícies web integrades. El principal inconvenient d'aquestes solucions és que sovint encara depenen de protocols propietaris o de la instal·lació de *plugins* externs (com els antics ActiveX o extensions específiques) per poder visualitzar el vídeo, cosa que limita la seva compatibilitat a navegadors o sistemes operatius concrets.

Aquesta dependència està desapareixent amb la tendència actual cap al **Plugin-less video**. L'aparició d'estàndards com **WebRTC** [1] ha suposat un punt d'inflexió, permetent la transmissió de fluxos multimèdia d'alta fidelitat i baixa latència de forma nativa en el navegador, eliminant les barreres de seguretat i instal·lació que imposaven les tecnologies anteriors. Cal destacar que el disseny *Plugin-free* respon a una necessitat imperativa de seguretat corporativa: actualment, la immensa majoria d'organitzacions i grans corporacions bloquegen per política interna la instal·lació d'executables o *plugins* a les estacions de treball dels operadors per evitar potencials vectors d'atac. En canvi, l'accés a través de navegadors web moderns està plenament permès i auditat, la qual cosa converteix la solució *Plugin-free* en un requisit de compliment de seguretat vital per a grans infraestructures.

D'altra banda, els líders de programari VMS (*Video Management System*) com Milestone o Genetec ofereixen solucions molt potents però amb arquitectures extremadament pesades i costos de llicenciament elevats, enfocades sovint a clients

amb gran capacitat de còmput local. La proposta de *Web Core View* es diferencia per ser una solució *embedded-first*, on el servidor web corre directament en el hardware de gravació.

1.3.2 Taula comparativa

A la Taula 1.1 es mostra una comparació entre l'aplicació web *legacy* anterior, els líders de mercat i la proposta de Web Core View.

Taula 1.1: Comparativa de solucions de mercat.

Característica	Web Legacy (ActiveX)	Market Leaders	Web Core View
Accessibilitat	Baixa (IE / ActiveX)	Mitjana (<i>Plugins</i>)	Alta (Navegador)
Manteniment	Difícil (Obsoleta)	Complex (Servidors)	Fàcil (Centralitzat)
Interfície (UX)	Obsoleta (anys 90)	Professional/Complexa	Moderna i Intuïtiva
Configuració	Estàtica	Propietària	Dinàmica (<i>Schema-driven</i>)

Font: Elaboració pròpia.

1.4 Anàlisi d'alternatives i justificació de la solució

Per garantir l'èxit i la sostenibilitat del projecte, s'han avaluat diferents camins tecnològics per als pilars fonamentals de programari, vídeo, concurrència i emmagatzematge del sistema:

1.4.1 Alternativa de Framework Frontend: Angular 19 vs. React / Vue

- **React / Vue:** Són llibreries extremadament populars i flexibles per a la creació d'interfícies. No obstant això, són arquitectures no opinades que requereixen la integració de múltiples llibreries de tercers per gestionar aspectes com el ruteig, formularis o seguretat. En un entorn de videovigilància professional embedded, això augmenta el risc de vulnerabilitats i encareix de manera crítica l'auditoria de seguretat.
- **Angular 19:** És un framework empresarial complet (*batteries-included*) que ofereix una estructura opinada, robusta i escrita nativament en TypeScript. Això garanteix que el codi s'organitzi de forma altament homogènia.
- **Justificació:** S'ha escollit **Angular 19** per la seguretat de tipus de TypeScript, el seu motor de formularis reactius natiu i, principalment, per la introducció de les ****Signals**** com a mecanisme natiu i eficient de gestió de l'estat reactiu, la qual cosa redueix de manera dràstica l'ús de CPU respecte a altres frameworks.

1.4.2 Alternativa de vídeo: Arquitectura Híbrida WebRTC i HLS vs. RTSP / RTMP

- **RTSP (Real-Time Streaming Protocol):** És el protocol de facto utilitzat internament per les càmeres IP i sistemes VMS clàssics. No obstant això, és completament incompatible de forma nativa amb els navegadors web actuals, obligant a utilitzar transcodificadors molt costosos o *plugins* externs propietaris en desús.
- **RTMP (Real-Time Messaging Protocol):** Protocol clàssic molt lligat a la tecnologia ja obsoleta d'Adobe Flash, actualment completament desapareguda de l'ecosistema web estàndard.
- **WebRTC i HLS:** Són els dos únics estàndards web moderns reconeguts pel W3C que permeten la reproducció de vídeo nativa (HTML5) de forma segura i sense cap mena d'extensió o control extern.
- **Justificació:** S'ha optat per una ****arquitectura híbrida complementària**** que explota el millor de cada protocol en lloc de triar-ne només un:
 1. **WebRTC:** S'utilitza de forma ****principal per al vídeo en directe****, on la latència ultra-baixa (< 300 ms) i el transport sobre UDP sota xifratge DTLS són indispensables per vigilar o respondre a amenaces immediates.
 2. **HLS (HTTP Live Streaming) [2]:** S'utilitza per a la ****reproducció de gravacions històriques****, on la ultra-baixa latència no és prioritària però es requereix un control molt precís de descàrrega per blocs (segments), commutació dinàmica de qualitat i la capacitat de fer desplaçaments lliures (**seek**) sobre l'històric mitjançant TCP. També actua com a **fallback** per al vídeo en directe si els tallafocs bloquegen el trànsit WebRTC.

1.4.3 Alternativa de Base de Dades per a Alarmes: SQLite3 vs. PostgreSQL / MongoDB

- **PostgreSQL / MySQL:** Són bases de dades relacionals molt potents, però funcionen com a serveis de sistema independents. Requereixen un consum elevat i constant de memòria RAM i CPU per mantenir el dimoni actiu, sent una sobrecàrrega inacceptable en dispositius embedded limitats.
- **MongoDB:** Base de dades documental NoSQL ideal per a grans volums de dades desestructurades, però amb una pila tecnològica pesada i requisits de recursos inassequibles per al maquinari de gravació local.
- **SQLite3:** És un motor de base de dades relacional *serverless*, transaccional (ACID) i auto-tingut que s'executa en el mateix procés del backend (in-process). Guarda tota la informació en un únic fitxer físic al disc.
- **Justificació:** S'ha seleccionat **SQLite3** com el motor de base de dades per al registre d'alarmes i logs del dispositiu, atès que ofereix un rendiment excel·lent

per a lectures i escriptures ràpides amb un consum residual de memòria (< 15 MB), essent ideal per a sistemes embedded Linux.

1.4.4 Alternativa de Còdec de Vídeo: H.264 i H.265 coexistents segons Maquinari i Càmera

- **H.264 (AVC):** És el còdec de compressió més universal i robust. És suportat pel 100% de les targetes gràfiques, dispositius mòbils i navegadors mitjançant descodificació accelerada per maquinari (GPU), garantint fluïdesa absoluta a costa d'un consum superior d'ample de banda.
- **H.265 (HEVC):** Algorisme de compressió d'última generació que permet reduir a la meitat l'espai d'emmagatzematge a disc i l'ample de banda de transmissió de vídeo per a la mateixa qualitat d'imatge. No obstant això, el seu suport a la web està limitat per llicències de patents, requerint que el client disposi d'un processador modern amb descodificació nativa integrada.
- **Justificació:** En lloc de descartar un dels dos còdecs, el sistema s'ha dissenyat amb una **estratègia dinàmica de coexistència**:
 1. Es fa servir **H.265** com a còdec d'alta prioritat quan la càmera IP física ho suporta i el dispositiu del client disposa de descodificació nativa per maquinari compatible al navegador, maximitzant així l'estalvi d'espai de disc als discs durs del gravador (clau per a instal·lacions amb desenes de càmeres de gran resolució).
 2. Es commuta de forma transparent cap a **H.264** si es detecta que la càmera és antiga o si el terminal de l'usuari és de gamma baixa i no disposa del motor de descodificació H.265 a la GPU, garantint així la portabilitat absoluta i la visualització sense retards ni sobrecàrrega de CPU.

1.4.5 Alternativa de configuració: Estàtica vs. Schema-driven

- **Formularis estàtics:** Programar una pantalla per a cada menú de configuració és més ràpid a curt termini però genera un deute tècnic insostenible, ja que cada canvi en el *firmware* obligaria a actualitzar el *frontend*.
- **Motor Schema-driven:** El *frontend* actua com un intèrpret que renderitza la interfície basant-se en un esquema JSON rebut del servidor.
- **Justificació:** S'ha optat per la solució **Schema-driven** per garantir la compatibilitat futura amb els nous models de gravador de Lanaccess sense haver de modificar el codi de l'aplicació web.

1.4.6 Alternativa de gestió d'estat: Redux vs. Angular Signals

- **Redux/NgRx:** Solucions robustes per a aplicacions gegantines, però amb un *boilerplate* elevat i un consum de recursos que pot penalitzar dispositius amb hardware limitat.
- **Angular Signals:** La nova eina nativa d'Angular 19 per a la gestió de la reactivitat granular [3].
- **Justificació:** S'ha seleccionat **Signals** per la seva lleugeresa i eficiència en el renderitzat. En un entorn *embedded* on la CPU s'ha de prioritzar per al vídeo, la reactivitat nativa de Signals permet prescindir de les comprovacions constants de *Zone.js*, optimitzant el rendiment global del sistema.

1.5 Solució proposada i valor afegit

Web Core View es proposa com una solució integral que aprofita les noves funcionalitats de rendiment d'**Angular 19**. L'elecció d'aquesta versió respon a la voluntat d'utilitzar tecnologies actualitzades i de llarg suport. Les aportacions clau de la solució proposada són:

- **Arquitectura Schema-driven:** La interfície interpreta un esquema JSON enviat pel gravador i genera automàticament els controls de configuració, assegurant compatibilitat amb qualsevol versió de hardware.
- **Gestió d'estat amb Signals:** L'ús d'**Angular Signals** representa un canvi de paradigma en la reactivitat. Aquesta funcionalitat permet una gestió granular dels canvis, optimitzant el rendiment del *frontend* i garantint una fluïdesa absoluta fins i tot amb múltiples fluxos de vídeo simultanis, evitant sobrecàrregues de CPU innecessàries al terminal del client.
- **Control de concurrència:** Implementació d'un servei de bloqueig en temps real per evitar conflictes d'edició entre administradors.
- **Arquitectura d'esdeveniments:** Ús de *WebSockets* (WSS) per rebre notificacions d'alarmes, notificacions del sistema i errors de forma immediata.
- **Desplegament autònom (single-NVR):** L'aplicació s'executa directament dins del propi videogravador com a servidor web *embedded*. Això permet que empreses petites amb un únic dispositiu puguin gestionar-lo de forma completa i autònoma des de qualsevol navegador, sense necessitat d'infraestructura centralitzada addicional ni de programari extern. Aquesta capacitat obre un nou segment de mercat per a Lanaccess que la plataforma *legacy* anterior no podia cobrir.

1.6 Objectius del projecte

1.6.1 Objectiu general

Dissenyar i desenvolupar una consola de gestió web modular, reactiva i d'alt rendiment que centralitzi la gestió tècnica i operativa dels dispositius Lanaccess, substituint la plataforma web *legacy* dels anys 90 i les seves tecnologies obsoletes (ActiveX) per una arquitectura i interfície completament modernes.

1.6.2 Objectius específics

- Desenvolupar un motor de configuració dinàmic basat en esquemes metadada.
- Integrar un mòdul de visualització híbrid compatible amb WebRTC i HLS (H264/H265).
- Implementar la gestió d'alarmes en temps real mitjançant comunicació bidireccional.
- Facilitar el manteniment remot (*firmware*, llicències i logs) via navegador.
- Dissenyar l'aplicació com una solució autònoma *embedded* capaç de funcionar directament en un únic gravador (*single-NVR*), sense dependència de servidors centralitzats, per cobrir el segment d'empreses petites.

1.7 Abast i Obstacles

1.7.1 Abast funcional

L'aplicació ha d'oferir un cicle complet de gestió: des de l'autenticació segura de l'usuari fins a la monitorització de càmeres en directe, la cerca de gravacions històriques i la configuració avançada de xarxa i paràmetres de sistema. El disseny ha de contemplar el desplegament autònom en un únic gravador (mode *single-NVR*) com a cas d'ús principal, garantint que tota la funcionalitat sigui operable sense infraestructura externa.

1.7.2 Obstacles detectats

S'han identificat reptes tècnics que afecten diferents actors:

- **Complexitat del vídeo natiu:** El repte de visualitzar H.265 sense *plugins*. Afecta al **Projectista** (recerca de llibreries) i a l'**Usuari final** (potencial latència).
- **Sincronització en temps real:** Requereix una gestió d'estat robusta perquè l'**Operador** vegi alarmes sense retard.
- **Seguretat i NGINX:** Configuració del **Proxy Invers** per centralitzar els microserveis sota un mateix origen i garantir polítiques de *Same-Origin*.

1.8 Actors implicats (Stakeholders)

1. **Usuari final (Operador):** Monitoritza càmeres i gestiona incidents.
2. **Administrador tècnic:** Configura el sistema i realitza manteniment.
3. **Lanaccess Telecom:** Proporciona el *hardware* i coneixement de negoci.
4. **Projectista (Estudiant):** Responsable únic de l'anàlisi i la implementació.
5. **Equip docent (Director i Ponent):** Supervisen el rigor acadèmic i tècnic.

Capítol 2

Gestió del projecte

En aquest capítol es descriu el marc metodològic utilitzat per al desenvolupament del projecte, la planificació temporal detallada de les tasques i l'anàlisi econòmica i de sostenibilitat del sistema.

2.1 Metodologia i eines de treball

El projecte es desenvolupa en un entorn professional dins de l'equip d'*Embedded* de Lanaccess. S'ha optat per una **metodologia àgil basada en Kanban**, utilitzant l'eina *Businessmap* per a la gestió visual del flux de treball. Aquesta elecció permet una gran flexibilitat i una resposta ràpida davant de canvis en els requeriments tècnics.

Els pilars del seguiment del projecte són:

- **Reunions de sincronització (Daily):** Sessions diàries per coordinar el progrés, identificar bloquejos i alinear el *frontend* amb l'estat del *firmware*.
- **Seguiment acadèmic i professional:** El director del TFG supervisa l'evolució tècnica diàriament, mentre que es manté una comunicació constant amb el tutor de la universitat per garantir el rigor acadèmic.
- **Gestió de tasques:** Control del cicle de vida de cada funcionalitat des de la columna *To Do* fins a *Done*.

Respecte a l'enginyeria de software, se segueixen els estàndards següents:

- **Control de versions:** Ús de **Git** amb el model **GitFlow** (branques *master*, *develop* i *feature*). El repositori es troba allotjat en un GitLab local.
- **Stack tecnològic:** Angular 19 (Signals i Standalone Components) per al *frontend* i implementacions en C++ 22 per a la comunicació de baix nivell en el *backend*.
- **Qualitat:** Aplicació de principis SOLID i arquitectura modular.

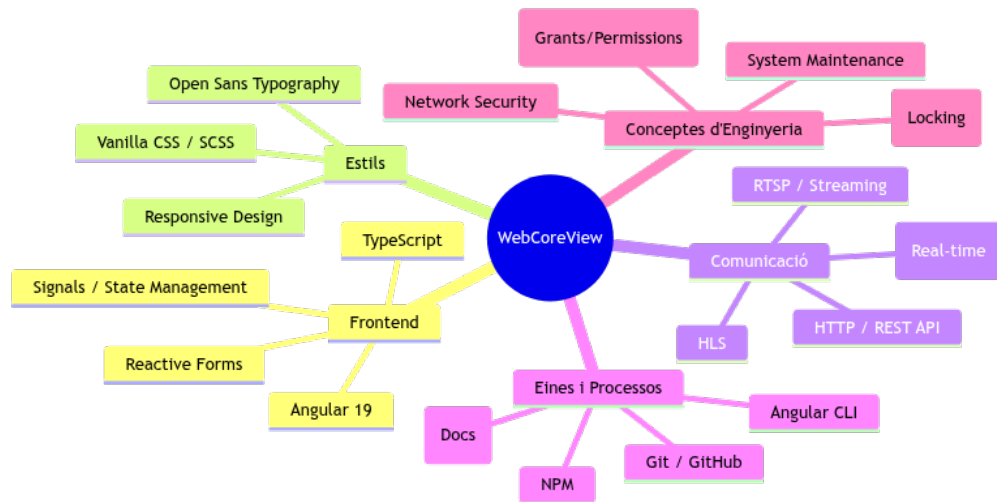


Figura 2.1: Mapa mental del stack tecnològic i d'enginyeria aplicat al projecte.
Font: Elaboració pròpia.

2.2 Planificació temporal

2.2.1 Descripció de les fases i tasques

El projecte s'estructura en sis fases lògiques pactades amb l'empresa:

1. **Gestió i Formació (GF):** Inclou l'autoformació en Angular 19 i la gestió administrativa de l'assignatura GEP.
2. **Mòdul d'Autenticació i Core (AC):** Desenvolupament del *login*, la *home page* i el sistema base de *logs* i alertes.
3. **Manteniment i Configuració Base (MC):** Gestió de llicències, actualització de *firmware* i integració del motor de base de dades.
4. **Alarmes i Sistema de Vídeo Directe (AV):** Implementació del motor d'alarmes via WSS i visualització en temps real.
5. **Mòdul de Gravacions i Configuració Avançada (GV):** Reproducció d'històrics (HLS) i motor de configuració dinàmica *Schema-driven*.
6. **Finalització i Memòria (FM):** Fase de correcció d'errors, repàs de qualitat i redacció final.

2.2.2 Recursos identificats

Per a l'execució del projecte s'han identificat i classificat els següents recursos:

Recursos humans (Rols)

Es defineixen els següents rols assumits pel desenvolupador al llarg del projecte:

- **Cap de Projecte (PM):** Responsable de la planificació temporal, coordinació amb Lanaccess i control econòmic.
- **Analista:** Dissenyador de l'arquitectura de capes, diagrames UML i definició de requisits.
- **Programador:** Codificador del frontend en Angular 19, configuració de NGINX i integració gRPC.
- **Tester / Validació:** Responsable del disseny de casos de prova, test de càrrega de vídeo i validació de seguretat.

Recursos materials

Comprèn exclusivament els elements físics no informàtics i d'infraestructura:

- **Espai físic:** Oficina de proves i laboratori de xarxes proveït per Lanaccess a les seves instal·lacions.
- **Mobiliari:** Escriptori d'oficina, cadira ergonòmica de treball i elements de papereria.

Recursos informàtics

Grup que agrupa tot el programari i el maquinari digital utilitzats pel projecte, evitant catalogar el software com a recurs material:

- **Maquinari (Hardware):** Ordinador portàtil (ASUS Expertbook Intel Core i7, 16GB RAM, anomenat PC) i videogravador de proves real (NVR Lanaccess, anomenat NVR) per a proves d'streaming WebRTC/HLS.
- **Programari de desenvolupament (Software):** IDE Visual Studio Code, entorn de versions Git, intèrpret de terminal Bash i compilador de LaTeX.
- **Tecnologies d'execució:** Framework Angular 19, reactivitat mitjançant Signals, protocol gRPC, connexions WebSockets (WSS) i llibreria multimèdia WebRTC natiu.

2.2.3 Estimació d'hores i Diagrama de Gantt

S'ha estimat una càrrega total de **540 hores** (18 ECTS). La Taula 2.1 resumeix el repartiment de tasques, les dependències de cicle de vida i els recursos associats.

Taula 2.1: Estimació d'hores i recursos utilitzats a cada tasca.

ID	Tasca	Dependència	Recursos	Hores
GF1	Formació Angular 19	-	Programador, PC	40
AC1	Login	GF1	Programador, Analista, PC	20
AC2	Home Page	AC1	Programador, Analista, PC	20
AV1	Core Alarm (WSS)	AC3	Programador, Analista, PM, PC	50
AV3	Càmeres Directe	AC2	Programador, Analista, PM, PC, NVR	50
GV2	System Config Avançat	MC2	Programador, Analista, PM, PC	60
FM2	Memòria TFG	-	PM, Analista, Programador, Tester, PC	80
TOTAL				540

Font: Elaboració pròpia.

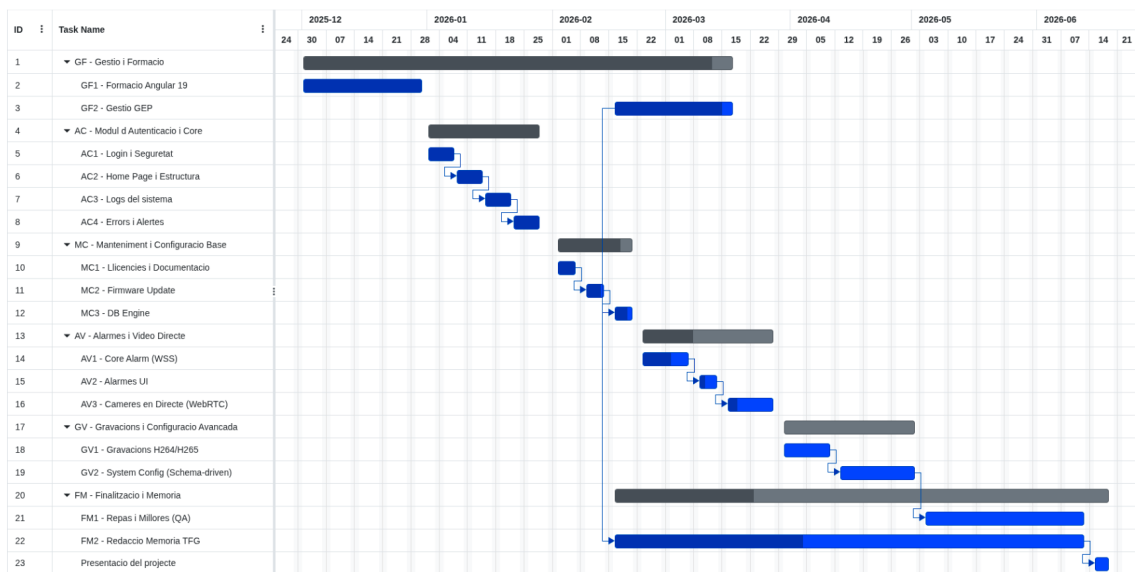


Figura 2.2: Diagrama de Gantt del projecte.

Font: Elaboració pròpia.

2.2.4 Gestió del risc i desviacions reals

Un desenvolupament tecnològic d'alta enginyeria en sistemes crítics i entorns de videovigilància corporativa comporta un conjunt de riscos que cal monitorar de manera constant. Per respondre de forma preventiva i correctiva, s'ha definit una matriu de gestió de riscos on es classifiquen segons la seva tipologia, la probabilitat d'ocurrència (Alta, Mitjana, Baixa) i l'impacte potencial en el cicle de vida del projecte (Alt, Mitjà, Baix), tal com es recull a la Taula 2.2.

A nivell de desviacions reals experimentades, durant l'execució del projecte s'han confirmat dos dels riscos recollits en la planificació:

1. **Complexitat en la integració del còdec H.265:** El comportament de la descodificació nativa en diferents navegadors ha requerit un buffer d'Angular molt més complex per garantir la fluïdesa.

Taula 2.2: Matriu de gestió de riscos i plans de contingència.

Tipus	Risc Identificat	Prob.	Imp.	Pla de Contingència / Acció Correctiva
Tecnològic	Incompatibilitat de descompressió de vídeo H.265	Mitjana	Alt	Optimització de la memòria intermèdia (<i>buffering</i>) a la capa de frontend. Sol·licitar ajustaments del format d'emissió H.265 directament a l'equip de firmware corporatiu.
Planificació	Corba d'aprenentatge d'Angular 19 i Signals	Baixa	Mitjà	Ús de sandboxes de prova reduïts, guies oficials de desenvolupament Angular i aprofitament d'arquitectures MVC heretades.
Tecnològic	Degradació del rendiment per configuracions anidades	Mitjana	Mitjà	Implementació d'algorismia de recorregut asíncron de dades al motor <i>schema-driven</i> per evitar bloquejar el thread del DOM.
Materials	Manca de disponibilitat de gravadors NVR de prova	Baixa	Alt	Creació d'un entorn de mock-up local mitjançant respostes JSON simulades que repliquin la interfície gRPC i D-Bus.
Humans	Descoordinació amb el departament de backend	Baixa	Mitjà	Establiment de contractes de dades rígids mitjançant definicions formalitzades de fitxers de missatges .proto a gRPC.

Font: Elaboració pròpia.

2. **Disseny arquitectònic del motor Schema-driven:** La complexitat teòrica del parseig de metadades JSON per garantir validacions complexes ha consumit més hores de l'estimat inicialment.

Aquestes desviacions i la seva gestió d'impacte es detallen en els següents apartats de rendiment i planificació econòmica.

2.2.5 Desviacions i impacte econòmic

Aquestes desviacions han tingut un impacte directe en el consum de recursos humans, especialment en els rols d'**Analista** i **Programador**. El temps addicional dedicat a la recerca i implementació del suport H.265 sense *plugins* ha suposat un increment aproximat de **45 hores** sobre la planificació inicial.

A nivell econòmic, aquest sobrecost teòric s'ha gestionat mitjançant el **marge de contingència del 10%** definit en el pressupost inicial. Tot i que el cost real del personal ha augmentat, el pressupost final no s'ha vist compromès gràcies a aquesta reserva, evitant així la necessitat de finançament addicional o de retallar funcionalitats crítiques de l'abast del projecte.

2.2.6 Estat actual del projecte

A data de la fita de seguiment (maig de 2026), el projecte es troba en un estat de desenvolupament del **85 %**. El desglossament per mòduls és el següent:

- **Mòdul d'Autenticació i Core:** 100 % (Completat).
- **Motor d'Alarmes i Vídeo Directe (WebRTC):** 100 % (Completat i validat).
- **Motor de Configuració Schema-driven:** 90 % (En fase de proves d'integració).
- **Mòdul de Gravacions (HLS):** 60 % (En fase d'optimització del buffer pel còdec H.265).
- **Memòria i Documentació:** 50 % (En procés).

Es preveu finalitzar el desenvolupament tècnic dins del termini previst, dedicant les últimes dues setmanes exclusivament al poliment de la UI i la redacció de l'annex tècnic.

2.3 Anàlisi econòmica

L'estimació econòmica s'ha realitzat assumint rols professionals i preus de mercat actuals, afegint un factor d'1,35 per cobrir la Seguretat Social.

Taula 2.3: Costos de recursos humans per rol.

Rol	Sou Brut/h (€)	Cost Real/h (€)	Hores	Total (€)
Project Manager	25,00	33,75	90	3.037,50
Analista	20,00	27,00	100	2.700,00
Programador	18,00	24,30	250	6.075,00
Tester	13,00	17,55	100	1.755,00
TOTAL CPA			540	13.567,50

2.3.1 Costos de personal

2.3.2 Costos totals i resum econòmic

Considerant tots els conceptes analitzats en la planificació econòmica, el pressupost global del projecte es pot resumir de la següent manera:

- **Cost de Recursos Humans (CPA):** 13.567,50 €.
- **Amortització del maquinari:** 210,00 €.
- **Costos indirectes (despeses generals d'oficina):** 1.150,00 €.
- **Marge de contingència (10 %):** 1.492,75 € (reserva preventiva calculada sobre la suma total de despeses).

D'aquesta manera, el pressupost final estimat del projecte ascendeix a **16.810,40 €**.

2.3.3 Control de gestió i seguiment de desviacions

Per garantir que el projecte no pateixi sobre costos inassumibles o retards estructurals crítics en la fita final, s'ha dissenyat un sistema de control de gestió basat en mètriques de rendiment. El seguiment es realitza de forma setmanal pel Cap de Projecte, mesurant l'evolució del treball mitjançant el càlcul d'indicadors de desviació acumulats.

Els indicadors aplicats són els següents:

- **Desviació de Terminis (DT):** Mesura el retard o avanç de calendari en hores reals d'execució:

$$DT = \text{Hores Planificades} - \text{Hores Reals}$$

Un valor negatiu de DT indica que s'està consumint més temps del planificat a causa de problemes de desenvolupament (com els descrits al buffer H.265).

- **Desviació del Pressupost (DP):** Avalua l'impacte financer sobre el cost del personal brut:

$$DP = \text{Cost Planificat} - \text{Cost Real}$$

i es calcula de forma desglossada de la següent manera:

$$DP = (H_{\text{plan}} - H_{\text{reals}}) \times \text{Cost/h}$$

- **Indicadors de Rendiment (EVM):** S'apliquen el CPI (*Cost Performance Index*) i l'SPI (*Schedule Performance Index*) per comparar l'evolució del valor obtingut enfront del cost real i planificat. Un valor inferior a 1,0 en ambdós indicadors marca alertes d'ineficiència temporal o sobrecost.

Per respondre ràpidament a aquests indicadors, s'estabilitza un protocol de llindars de tolerància i d'accions correctives immediates:

- **Llindar Verd (Desviació $< 5 \%$):** Situació controlada. Qualsevol lleugera desviació d'hores es compensa en les tasques posteriors més flexibles, especialment en les de documentació i proves transversals.
- **Llindar Groc (Desviació 5% - 10%):** Alerta moderada. Requereix l'optimització de codi existent per evitar desbordaments temporals de rols d'Analista o Programador. Si hi ha desviació financera s'inicia el consum parcial del marge de contingència previst del 10% .
- **Llindar Vermell (Desviació $> 10 \%$):** Alerta crítica de projecte. Aquest cas s'ha activat realment davant de la complexitat acumulada en H.265 i el motor *schema-driven*. Seguint el protocol definit, es reavalua immediatament l'abast del projecte i s'apliquen mesures correctives dràstiques. En aquest cas real, la mesura correctora clau ha estat la decisió de dissenyar una versió simplificada dirigida a petites instal·lacions (*single-NVR*). Això permet aïllar els retards del desplegament centralitzat massiu, concentrant les hores i els recursos de personal en un lliurament funcional de qualitat que assegura complir el pressupost global del projecte sense necessitat de refinançament.

2.4 Tancament del projecte i desviacions finals

Un cop completat el cicle de desenvolupament i la redacció de la present memòria, es procedeix a realitzar el tancament del projecte. Aquest tancament té com a objectiu analitzar de forma retrospectiva les desviacions ocorregudes tant en la planificació temporal com en la gestió del pressupost, justificant les mesures correctores aplicades.

2.4.1 Tancament temporal: Planificació prevista vs. real

Tot i que la càrrega de treball total s'ha mantingut en l'ordre de les **540 hores** globals definides durant la fase d'iniciativa GEP (corresponents a 18 crèdits ECTS), la distribució interna de les tasques ha patit desviacions significatives a causa de la complexitat tècnica del sistema.

La Taula 2.4 recull la comparativa de les hores previstes en la planificació inicial enfront de les hores realment consumides per cada una de les fases principals del projecte.

Com s'observa en els indicadors, les fases de desenvolupament tècnic pur (**AV** i **GV**) han patit una desviació a l'alça acumulada de **+45 hores**. Aquesta desviació es justifica per dos motius principals:

Taula 2.4: Comparativa de la planificació temporal prevista enfront del temps real d'execució.

Fase	Descripció	Hores Previstes	Hores Reals	Desviació (h)
GF	Gestió i Formació	40	35	-5
AC	Mòdul d'Autenticació i Core	60	50	-10
MC	Manteniment i Configuració Base	70	65	-5
AV	Alarmes i Vídeo Directe (WebRTC)	120	145	+25
GV	Gravacions i Configuració Avançada	170	190	+20
FM	Finalització i Memòria	80	55	-25
TOT.		540	540	0

Font: Elaboració pròpia.

1. **El buffer i reproducció H.265 (fase AV):** La implementació i proves de descompressió de vídeo en temps real natiu sense dependència de *plugins* sota diferents navegadors moderns va presentar problemes insospitats de desbordament del buffer HTML5, forçant un desenvolupament asíncron addicional.
2. **El motor de parseig del Schema-driven (fase GV):** El recorregut recursiu i validació recursiva de JSONs amb validacions complexes sobre camps anidats va resultar arquitectònicament més complex de l'estimat.

Per mantenir la data de lliurament immutable i respectar les 540 hores màximes establertes, es van aplicar les següents accions correctores en les fases de finalització:

- **Optimització de la redacció (FM):** L'elaboració de la memòria es va realitzar de forma altament paral·lela i simplificada aprofitant els diagrames de Mermaid de la documentació tècnica de l'empresa, estalviant **25 hores** d'escriptura repetitiva.
- **Retall d'abast no essencial:** Es van automatitzar les proves unitàries amb perfilador natiu en lloc de desplegar tests e2e extremadament densos que haguessin sobrecarregat les fases finals.

2.4.2 Tancament econòmic: Pressupost estimat vs. final

La variació de les hores a nivell de tasques de desenvolupament ha repercutit directament sobre els costos associats als rols de personal corporatiu brut (**CPA**). No obstant això, el pressupost global del projecte no s'ha vist alterat.

La Taula 2.5 detalla la comparativa econòmica per cada un dels rols humans definits.

A causa de les desviacions de desenvolupament indicades anteriorment, s'ha hagut de destinar més temps d'enginyeria als rols d'**Analista** (+20 h) i **Programador** (+35 h), el que ha provocat un lleuger desviament brut del personal de **+101,25 €**.

Aquest sobrecost s'ha aconseguit absorbir de forma directa mitjançant l'ús de la partida preventiva del **Marge de Contingència (10 %)**, que estava pressupostada

Taula 2.5: Comparativa del cost de recursos humans estimat enfront de l'executat real.

Rol	Cost Estimat (€)	Hores Reals	Cost Real (€)	Desviació (€)
Project Manager	3.037,50	70	2.362,50	-675,00
Analista	2.700,00	120	3.240,00	+540,00
Programador	6.075,00	285	6.925,50	+850,50
Tester	1.755,00	65	1.140,75	-614,25
CPA	13.567,50	540	13.668,75	+101,25

Font: Elaboració pròpia.

inicialment en **1.492,75 €**. Un cop alliberats els fons restants d'aquesta partida i consolidada la resta de costos immutables (amortitzacions de maquinari a **210,00 €** i costos indirectes d'oficina mantinguts en **1.150,00 €**), el pressupost total final executat s'ha tancat en **15.128,75 €** (enfront dels 16.810,40 € inicialment estimats incloent el total del marge sense consumir).

D'aquesta manera, el projecte es tanca amb un estalvi net de **1.681,65 €** respecte a l'estimació de despesa màxima prevista pel negoci, demostrant una eficiència econòmica òptima.

2.5 Anàlisi de sostenibilitat i implicacions ètiques

L'avaluació global del projecte requereix analitzar-ne l'impacte des de quatre dimensions fonamentals:

- **Dimensió Econòmica:** La solució *Schema-driven* permet adaptar el software a nous models de gravadors sense cost addicional d'enginyeria frontend, allargant la vida útil del producte i reduint el deute tècnic de l'empresa. El model *single-NVR* democratitza l'accés a sistemes professionals per a petites empreses.
- **Dimensió Ambiental:** L'ús d'una plataforma web remota accessible des de qualsevol navegador redueix dràsticament els desplaçaments físics de tècnics de manteniment a les instal·lacions del client, minimitzant directament la petjada de carboni associada a la flota de vehicles.
- **Dimensió Social:** Millora significativament la interfície de treball dels operadors de seguretat. S'ha reduït la càrrega cognitiva gràcies a un disseny clar i reactiu, fet que disminueix l'estrès operatiu i la fatiga visual en situacions d'incident crític o torns de monitorització perllongats.
- **Implicacions Ètiques:** Tractant-se d'un sistema de videovigilància, el projecte ha integrat el principi de *Privadesa per Disseny*. L'arquitectura assegura que només els usuaris amb privilegis explícits puguin visualitzar el vídeo. El compliment de les directives RGPD s'aborda tècnicament garantint l'esborrat de dades passats 30 dies i la no persistència d'imatges o dades biomètriques al terminal de l'operador. A més, s'evita la inclusió de perfils automatitzats

opacs, garantint que les decisions de seguretat recaiguin sempre sobre el judici humà sota una estricta traçabilitat auditable (*Audit Log*).

Capítol 3

Especificació de Requisits

L'especificació de requisits és la pedra angular del projecte *Web Core View*. En tractar-se d'un sistema de gestió per a dispositius de seguretat crítica, la definició de les funcionalitats no només s'ha centrat en l'experiència d'usuari (UX), sinó també en la robustesa de la comunicació asíncrona i el desacoblament entre el hardware i la interfície.

En aquest capítol es detallen els actors, les històries d'usuari mitjançant la metodologia *Product Backlog*, i l'especificació detallada dels requisits funcionals i no funcionals.

3.1 Metodologia d'extracció de requisits

Els requisits aquí exposats s'han obtingut mitjançant un procés iteratiu amb l'equip d'Embedded de Lanaccess. S'han realitzat sessions d'anàlisi sobre l'aplicació web *legacy* existent — basada en tecnologies obsoletes com ActiveX — i entrevistes amb tècnics de camp per identificar els “punts de dolor” i les limitacions tecnològiques de la plataforma anterior.

3.2 Identificació d'actors

S'han definit tres actors principals, classificant-los segons el seu nivell d'interacció i privilegis dins del sistema:

Administrador de Sistema: Actor amb privilegis totals sobre el videogravador.

El seu objectiu és la posada en marxa de l'equip i el manteniment correctiu. Les seves responsabilitats inclouen:

- Configuració de paràmetres de xarxa (IP, DNS, Gateways).
- Gestió del cicle de vida del hardware (Reinícis, actualització de firmware).
- Administració de comptes d'usuari i permisos.

Operador: Usuari final que utilitza la plataforma per a la monitorització.

- Visualització de vídeo en temps real (*Live*).

- Navegació i exportació de gravacions històriques.
- Supervisió i silenciament d'alarmes en temps real.

Videogravorador (System Actor): Malgrat ser un element de hardware, en aquesta arquitectura actua com un usuari actiu que:

- Dicta la estructura de la interfície mitjançant esquemes JSON.
- Notifica esdeveniments de forma proactiva via WebSockets.
- Gestiona el control de concurrència per evitar col·lisions de dades.

3.3 Històries d'Usuari (User Stories)

S'utilitzen les *User Stories* per definir el valor que cada funcionalitat aporta a l'usuari final. Cada història inclou els seus criteris d'acceptació.

3.3.1 Mòdul de Visualització i Reproducció de Vídeo

US-01: Streaming d'alta fidelitat

Com a Operador, **vull** veure el vídeo de qualsevol càmera connectada, **per tal de** vigilar l'espai protegit en temps real.

- *Criteri 1:* La latència punta a punta (glass-to-glass) ha de ser inferior a 300ms.
- *Criteri 2:* El sistema ha de detectar automàticament si el navegador suporta WebRTC o ha de commutar a HLS.

US-02: Gestió de Mosaic

Com a Operador, **vull** visualitzar fins a 4 càmeres simultàniament, **per tal de** tenir una visió global de la instal·lació.

US-04: Consulta de gravacions històriques

Com a Operador, **vull** consultar i reproduir el vídeo gravat del passat mitjançant una línia de temps (*timeline*) gràfica, **per tal de** revisar incidents de seguretat que s'hagin produït prèviament.

- *Criteri 1:* El reproductor ha de permetre saltar a qualsevol punt temporal passat amb una precisió de 1 segon a través del lliscador de la línia de temps.
- *Criteri 2:* La reproducció de vídeo històric s'ha de transmetre mitjançant fragments HLS generats a petició pel gravador.

3.3.2 Mòdul de Gestió d'Alarmes

US-05: Recepció d'alarmes en temps real

Com a Supervisor, **vull** rebre notificacions d'alarmes de forma immediata, **per tal d'**actuar ràpidament davant de qualsevol incidència de seguretat.

- *Criteri 1:* L'alarma s'ha de mostrar a la interfície web en menys d'1 segon des de la seva detecció en el maquinari.

US-06: Configuració de regles d'alarma

Com a Administrador, **vull** configurar i programar regles de detecció d'alarmes per a cada càmera, **per tal d'**automatitzar la detecció d'esdeveniments.

3.3.3 Mòdul de Configuració Dinàmica i Xarxa

US-03: Configuració sense recàrrega de codi

Com a Administrador, **vull** que els nous paràmetres afegits al firmware apareguin a la web sense haver de reprogramar el frontend.

- *Criteri 1:* El frontend ha d'interpretar l'esquema JSON i renderitzar el component visual adequat de forma dinàmica (input, select, slider).

US-07: Configuració de xarxa i IP

Com a Administrador, **vull** configurar els paràmetres de xarxa del dispositiu (adreça IP, màscara, ports), **per tal de** garantir la connectivitat del sistema.

US-08: Gestió d'usuaris i permisos

Com a Administrador, **vull** gestionar els perfils d'usuari i els seus permisos d'accés (grants), **per tal de** controlar quines accions i vistes pot utilitzar cadascú.

3.3.4 Mòdul de Manteniment

US-09: Monitoratge dels discs durs

Com a Tècnic, **vull** supervisar l'estat i la capacitat dels discs durs de gravació, **per tal d'**assegurar-me que no es perden gravacions històriques.

US-10: Descàrrega i consulta de logs

Com a Tècnic, **vull** cercar i descarregar els registres d'esdeveniments del sistema, **per tal de** diagnosticar errors de funcionament o auditar l'ús de la plataforma.

3.3.5 Flux d'Interacció (User Journey)

Per tal d'il·lustrar de quina manera els operadors de seguretat utilitzen aquest conjunt d'històries d'usuari per respondre de forma pràctica i integrada a un incident de seguretat real, es presenta el diagrama de User Journey (Figura 3.1).

3.4 Requisits Funcionals (RF)

A continuació es detallen els requisits tècnics dividits per blocs funcionals.

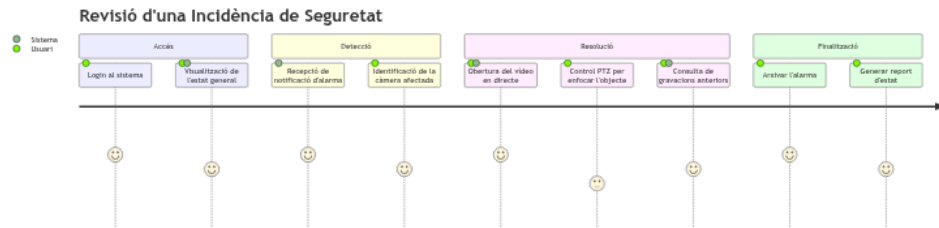


Figura 3.1: Diagrama de flux d'interacció de l'usuari (User Journey) en la resolució d'una incidència de seguretat.

Font: Elaboració pròpia.

3.4.1 RF-1: Gestió de l'Accés i Seguretat

- **RF-1.1 Autenticació:** El sistema ha de validar les credencials contra el servei *Core Legacy*.
- **RF-1.2 Sessió Persistent:** Mantenir la sessió mitjançant un mecanisme de *Keep-alive* que refresqui el token cada 60 segons.
- **RF-1.3 Control de Concurrència (Config Lock):** El sistema ha de demanar un bloqueig d'escriptura quan un administrador entri en mode edició. Si un altre usuari intenta editar, el sistema mostrarà un avís de "Només Lectura".

3.4.2 RF-2: Visualització Multimèdia

- **RF-2.1 Protocol WebRTC:** Implementació de la negociació SDP (*Session Description Protocol*) per a vídeo en directe.
- **RF-2.2 Protocol HLS:** Generació de fragments de vídeo per a la reproducció de gravacions històriques i fallback de seguretat.
- **RF-2.3 Línia de Temps Històrica (Timeline):** Interfície gràfica interactiva per a la cerca i consulta de gravacions passades amb selecció de franja horària.

3.4.3 RF-3: Motor de Configuració Dinàmica

Aquest és el requisit funcional més complex i innovador del sistema.

- **RF-3.1 Interpretació de l'Esquema:** El sistema ha de processar un fitxer JSON de definició. Per tal d'il·lustrar la capacitat del motor dinàmic (*schema-driven*) de desacoblar la interfície del frontend respecte de la lògica de baix nivell del gravador, es presenta a continuació un fragment d'esquema JSON de configuració avançada real enviat pel servei de vídeo. Aquest esquema defineix els tipus de transport tolerats pel maquinari de visualització d'una càmera IP física (utilitzant estructures d'enumeracions) i la restricció d'adreçaments de multidifusió (multicast) utilitzant expressions regulars complexes (*regex*) que el motor del frontend compilarà i validarà dinàmicament.

```
1 {
2   "media.video.{streamId}.receiver.transport": {
3     "_type": "list",
4     "_allowedValues": ["unicast", "multicast", "tcp"],
5     "_description": "Tipus de transport de xarxa per a la recepció
de vídeo RTSP",
6     "_default": "unicast"
7   },
8   "media.video.{streamId}.multicast.address": {
9     "_type": "string",
10    "_description": "Adreça IP de multidifusió assignada al perfil
de streaming",
11    "_maxLength": 15,
12    "_regex": "^(22[4-9]|23[0-9])
\\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)
\\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)
\\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$",
13    "_default": "0.0.0.0"
14  },
15  "media.video.{streamId}.multicast.port": {
16    "_type": "integer",
17    "_min": 1024,
18    "_max": 65535,
19    "_description": "Puerto de destinació per a la transmissió
Multicast de dades de vídeo",
20    "_default": 30010
21  }
22 }
```

Listing 3.1: Exemple d'esquema avançat JSON de metadades per a IP Video amb restriccions regex.

3.5 Requisits No Funcionals (RNF)

Els RNF defineixen les qualitats del sistema i les restriccions tecnològiques.

3.5.1 RNF-1: Rendiment i Eficiència

- **RNF-1.1 Utilització de CPU:** Atès que el videogravador té recursos limitats, el frontend no ha de fer més d'una petició de configuració per minut si no hi ha canvis de l'usuari.
- **RNF-1.2 Angular Signals:** La reactivitat de la interfície s'ha de basar en *Angular Signals* per minimitzar les operacions de *Zone.js* i evitar el re-renderitzat innecessari del DOM, especialment crític amb vídeos actius.

3.5.2 RNF-2: Robustesa i Disponibilitat

- **RNF-2.1 Tolerància a talls de xarxa:** El WebSocket ha d'intentar una reconexió exponencial (re-try) en cas de pèrdua de connectivitat.
- **RNF-2.2 Fallback de Vídeo:** Si el port WebRTC (UDP 10000-20000) està bloquejat per firewall, el reproductor ha de commutar a TCP-HLS en menys de 5 segons.
- **RNF-2.3 Concurrencia i Integritat (Pessimistic Locking):** Per garantir la consistència de les dades del sistema i evitar escriptures concurrents conflictives entre diferents usuaris administradors, s'exigeix un mecanisme de bloqueig pessimista (*Pessimistic Locking*) en l'adquisició dels formularis de configuració activa. Cap usuari podrà modificar paràmetres del sistema si un altre usuari posseeix el *lock* exclusiu d'aquell mòdul.

3.5.3 RNF-3: Compatibilitat i Portabilitat

- **RNF-3.1 Multi-navegador:** Compatible amb les últimes dues versions de Google Chrome, Mozilla Firefox i Microsoft Edge.
- **RNF-3.2 Responsive Design:** La interfície ha de ser plenament operativa en resolucions des de 1024px d'amplada (tablets de manteniment) fins a 4K (centres de control).

3.6 Casos d'Ús detallats

Per aprofundir en la funcionalitat del sistema, es presenta el diagrama de casos d'ús general (Figura 3.2) i l'especificació detallada dels fluxos crítics.

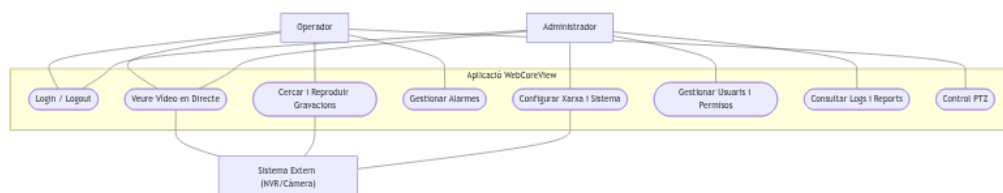


Figura 3.2: Diagrama UML de Casos d'Ús.

3.6.1 UC-01: Gestió de l'actualització de firmware

Aquest cas d'ús és crític per al manteniment del parc de gravadors.

ID i Nom	UC-01: Actualització remota de firmware
Actor	Administrador
Descripció	L'usuari carrega un binari de firmware per actualitzar les capacitats del dispositiu.
Precondicions	Usuari autenticat com administrador. El gravador ha de tenir més d'un 10% d'espai lliure a la partició de sistema.
Flux Principal	1. L'usuari accedeix al mòdul de Manteniment. 2. L'usuari selecciona un fitxer <i>.bin</i> local. 3. El frontend valida l'extensió i mida del fitxer. 4. S'inicia la pujada via POST multipart. 5. El sistema mostra una barra de progrés real basada en els esdeveniments del backend. 6. Un cop finalitzat, el gravador inicia el procés de <i>flashing</i> i es reinicia.
Flux Alternatiu	5a. Si la connexió es talla durant la pujada, el frontend informa de l'error i permet reintentar des del punt de fallada si el servidor ho suporta.
Postcondició	L'equip es reinicia. La interfície web detecta el tall i mostra una pantalla d'espera fins que l'equip torna a estar en línia. En cas d'actualització crítica amb fallida de verificació del checksum o corrupció del binari, el backend garanteix la recuperació automàtica commutant de forma transparent a la partició de firmware previ (<i>dual-boot rollback</i>) per assegurar la integritat i operativitat del dispositiu.

Taula 3.1: Especificació detallada del Cas d'Ús UC-01.

3.6.2 UC-02: Consulta de gravacions mitjançant Timeline

3.7 Restriccions del sistema

El desenvolupament s'ha de cenyir a les següents limitacions:

- **Seguretat:** No es permet l'ús de galetes (*cookies*) per a l'emmagatzematge de dades de sessió per evitar atacs CSRF; s'ha d'usar *LocalStorage* amb protecció de rutes.
- **Same-Origin Policy:** Les peticions només poden ser acceptades si provenen del mateix origen que serveix l'aplicació Angular (configurat al NGINX).
- **Llicència:** El sistema ha de verificar que la llicència del hardware permet la visualització de vídeo abans d'obrir els ports de streaming.

ID i Nom	UC-02: Cerca de vídeo històric
Actor	Operador
Flux Principal	1. L'usuari activa el mode "Playback". 2. El frontend sol·licita l'índex de gravacions per a una data concreta. 3. El motor de renderitzat dibuixa una línia de temps amb colors diferents segons el tipus de gravació (contínua, per alarma). 4. L'usuari fa clic en un punt de la línia. 5. El reproductor HLS inicia la càrrega del fragment corresponent a aquell <i>timestamp</i>.

Taula 3.2: Especificació detallada del Cas d'Ús UC-02.

3.8 Marc normatiu i regulacions

Atesa la naturalesa del projecte, que gestiona fluxos de vídeo en temps real, reproduccions d'imatges de persones físiques i dades de configuració de dispositius de seguretat crítics, el disseny i desenvolupament de *Web Core View* s'ha cenyit als estàndards legals i regulacions tècniques nacionals i europees:

3.8.1 Protecció de Dades de Caràcter Personal (RGPD i LOPDGDD)

El sistema s'adapta al **Reglament General de Protecció de Dades (RGPD - Reglament UE 2016/679)** [4] i a la legislació de transposició espanyola, la **Llei Orgànica 3/2018 de Protecció de Dades Personals i Garantia dels Drets Digitals (LOPDGDD)** [5]. Especialment, el disseny s'alinea amb l'**Article 22 de la LOPDGDD**, que regula específicament el tractament d'imatges amb finalitats de videovigilància, sota el principi fonamental de ****Privadesa per Disseny (Privacy by Design)****:

- **Minimització de dades:** No s'emmagatzemen imatges, captures ni dades personals de forma persistent en el navegador, exceptuant el *token* de sessió protegit en el *LocalStorage*.
- **Seguretat d'accés:** L'accés a les imatges està rígidament controlat per rols d'usuari i privilegis definits en el *backend*, assegurant que només els operadors explícitament autoritzats puguin visualitzar el vídeo en directe i els històrics.

3.8.2 Seguretat en la comunicació i Xifratge

Per garantir la integritat i la confidencialitat de les dades, i evitar atacs de tipus interceptació de xarxa o *Man-in-the-Middle* (MitM), s'obliga a l'ús de protocols criptogràfics segurs:

- Totes les peticions a l'API REST es realitzen sota el protocol de transport xifrat **HTTPS**.

- La comunicació asíncrona d'alarmes es realitza mitjançant canals de **WebSockets Segurs (WSS)**.
- La transmissió de fluxos de vídeo WebRTC utilitza xifratge d'extrem a extrem mitjançant el protocol **DTLS** per al canal de control i **SRTP** per a la seguretat en els paquets multimèdia.

3.8.3 Llei de Seguretat Privada i Norma UNE-EN 62676

El disseny tècnic actua com un element facilitador perquè les entitats de seguretat compleixin amb la **Llei 5/2014 de Seguretat Privada** a Espanya, així com amb l'estàndard tècnic europeu **UNE-EN 62676** sobre sistemes de videovigilància per a aplicacions de seguretat:

- **Control de caducitat i esborrat automàtic:** El sistema permet configurar de manera dinàmica les polítiques d'emmagatzematge per garantir que les gravacions es sobreescrueixen automàticament en un període màxim de **30 dies**, complint rigorosament amb el límit d'emmagatzematge fixat per la Llei de Seguretat Privada i l'Agència Espanyola de Protecció de Dades (AEPD), excepte en cas de requeriment judicial.
- **Registre d'auditoria (Audit Log):** L'aplicació compta amb un servei de registre d'activitat inalterable que monitoritza i audita les accions dels usuaris (qui ha visualitzat quina càmera, en quines dates i hores, i si s'han realitzat descàrregues de vídeo), assegurant la traçabilitat exigida per a la custòdia d'evidències gràfiques.

Malgrat aquestes facilitats tècniques, el compliment final de la normativa depèn de la configuració i l'ús real que l'entitat de seguretat privada faci del sistema en la instal·lació del client.

Capítol 4

Arquitectura i Disseny

L'arquitectura de *Web Core View* s'ha dissenyat sota els principis de modularitat, escalabilitat i eficiència en el consum de recursos. Atès que el programari s'executa en un entorn *embedded* (el videogravador), s'ha optat per una arquitectura desacoblada on el *frontend* assumeix la major part de la lògica de presentació per tal d'alliberar la CPU del hardware per a les tasques crítiques de seguretat.

4.1 Arquitectura Global del Sistema

El sistema es basa en un model de microserveis interns coordinats dins del sistema operatiu del gravador. La comunicació amb l'exterior es centralitza a través d'un punt d'entrada robust que gestiona el trànsit de dades i el flux de vídeo.

A nivell de desplegament, el sistema segueix una distribució de components modular, detallada en el diagrama de desplegament de la Figura 4.1.

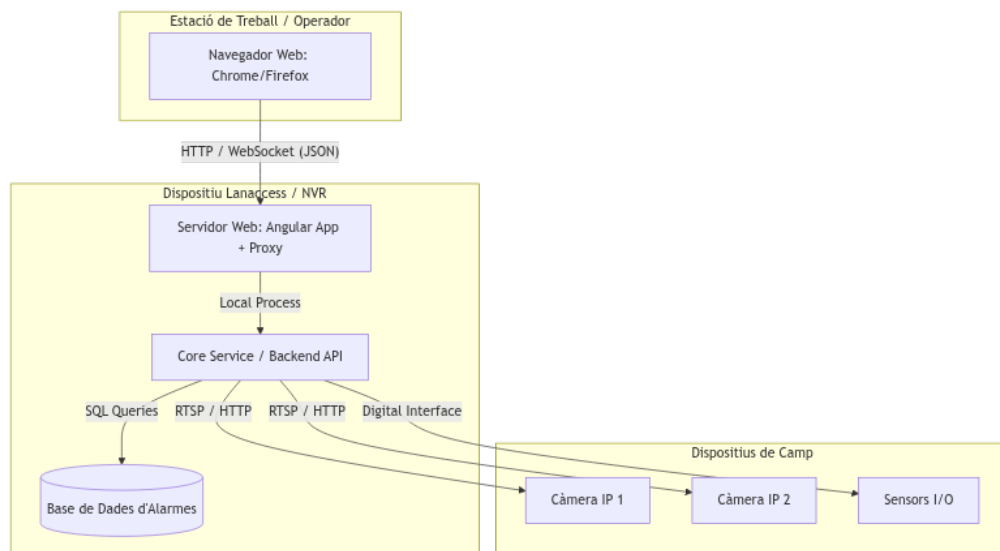


Figura 4.1: Diagrama de desplegament físic i lògic del sistema WebCoreView.

Font: Elaboració pròpia.

4.1.1 Capa de comunicació i Microserveis

Com es detalla a la Figura 4.1, la interacció entre la interfície i el hardware es divideix en tres canals principals:

- **Proxy Invers (NGINX):** Actua com a porta d'enllaç única. La seva funció és gestionar el xifratge TLS, resoldre les polítiques de *Cross-Origin Resource Sharing* (CORS) i redirigir les peticions cap als serveis interns corresponents.
- **API REST (HTTP):** S'utilitza per a operacions *stateless* i síncrones via mòdul *Core Legacy*. Gestiona la consulta i l'enviament de paràmetres de configuració.
- **Canal de Temps Real (WebSockets):** Mitjançant el protocol WSS, els mòduls *Core Notifier* i *Core Alarm* envien esdeveniments de forma asíncrona. Això permet que la interfície reaccioni instantàniament a una pèrdua de vídeo o a una activació de sensor sense necessitat de fer *polling* constant.

Internament, els serveis del backend embedded es comuniquen mitjançant **gRPC** [6] per a una transferència de dades binària d'alt rendiment i utilitzen un bus de missatgeria **D-Bus** per a la coordinació d'estats a nivell de sistema operatiu. Aquesta naturalesa heterogènia de la pila de comunicació del gravador planteja un repte de disseny crític: el navegador del client web no pot consumir directament gRPC binari ni escoltar senyals del bus D-Bus de forma nativa.

Per resoldre aquest desacoblament, s'ha dissenyat un mecanisme de ****bridging** de protocols (Protocol Bridging)** centralitzat en el servei de notifikacions (*Core Notifier*) del gravador. Aquest servei actua com a intermediari actiu (*gateway*): es subscriu als esdeveniments de sistema (com pèrdues de vídeo o activació de relés) a través del bus de sistema D-Bus i de canals gRPC, tradueix aquestes estructures de baix nivell en missatges lleugers en format JSON, i finalment les transmet en temps real a través del canal de **WebSockets** (WSS) [7] cap al client Angular. Aquest disseny de traducció bidireccional garanteix que el frontend pugui respondre immediatament a qualsevol esdeveniment de seguretat de forma asíncrona sense requerir costoses connexions binàries natives ni carregar la CPU del client.

4.1.2 Seguretat Defensiva contra vectors d'atac OWASP

Com a requisit no funcional de seguretat crítica, el frontend s'ha dissenyat per mitigar de forma activa les principals vulnerabilitats del catàleg OWASP Top 10. S'ha eliminat l'ús de cookies per a la persistència de sessió, utilitzant en el seu lloc capçaleres portadores (Bearer tokens) injectades dinàmicament a la memòria de l'aplicació per evitar atacs de tipus Cross-Site Request Forgery (CSRF).

Adicionalment, a la capa de NGINX, s'ha unificat l'origen de l'aplicació i l'API REST sota un mateix servidor de recursos, el que permet aplicar polítiques de Same-Origin estrictes i capçaleres de seguretat Content Security Policy (CSP) per immunitzar el sistema contra injeccions Cross-Site Scripting (XSS) i intents de Clickjacking.

```

1 # Configuració de seguretat NGINX per a l'entorn embedded de Web
  Core View
2 server {
3     listen 443 ssl http2;
4     server_name localhost;
5
6     # Certificats SSL i xifratge TLS fort
7     ssl_certificate /etc/nginx/ssl/server.crt;
8     ssl_certificate_key /etc/nginx/ssl/server.key;
9     ssl_protocols TLSv1.2 TLSv1.3;
10
11    # Capçaleres de seguretat defensiva (OWASP Mitigation)
12    add_header Content-Security-Policy "default-src 'self'; script-
src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
data:; connect-src 'self' wss:;" always;
13    add_header X-Frame-Options "DENY" always;
14    add_header X-Content-Type-Options "nosniff" always;
15    add_header Referrer-Policy "strict-origin-when-cross-origin"
always;
16
17    # Enrutament de l'aplicació SPA Angular (Frontend)
18    location / {
19        root /var/www/web-core-view;
20        try_files $uri $uri/ /index.html;
21    }
22
23    # Proxy Invers i Bridging per a WebSockets d'esdeveniments
24    location /ws/ {
25        proxy_pass http://localhost:8080;
26        proxy_http_version 1.1;
27        proxy_set_header Upgrade $http_upgrade;
28        proxy_set_header Connection "Upgrade";
29        proxy_set_header Host $host;
30    }
31 }

```

Listing 4.1: Configuració de seguretat NGINX per a l'entorn embedded de Web Core View.

4.1.3 Estratègia de Fallback i robustesa del canal de vídeo

La integració de WebRTC com a mètode de visualització de vídeo de baixa latència (< 300 ms) requereix un mecanisme d'alta disponibilitat que respongui davant de xarxes corporatives on el trànsit UDP pugui estar completament bloquejat pels tallafocs de seguretat. Per a aquest propòsit, s'ha dissenyat un sistema de *fallback* transparent a HLS [2].

El client web inicia la negociació SDP mitjançant la referència del protocol WebRTC W3C [8]. Si el navegador detecta que l'estat de la connexió ICE (*Interactive Connectivity Establishment*) transita cap a *failed* o *disconnected* després d'un *timeout* de seguretat de 5 segons, s'activa automàticament la lògica de degradació de servei. En aquest instant, es tanquen de manera neta els descriptors de WebRTC i s'inicialitza la reproducció d'un flux de vídeo HLS a través del Proxy Invers NGINX

(TCP, port 443 xifrat), assegurant que l'operador continuï visualitzant la càmera sense interrupció.

4.2 Disseny del Frontend (Angular 19)

L'elecció d'Angular 19 no és casual; el *framework* proporciona una estructura que facilita el manteniment a llarg termini de l'aplicació.

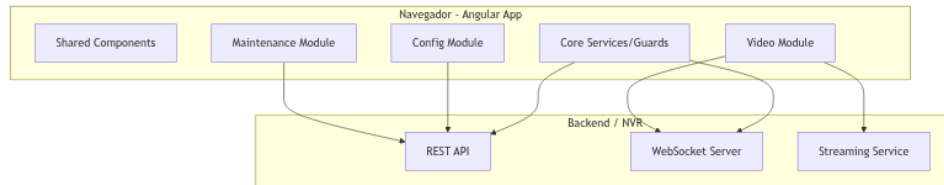


Figura 4.2: Diagrama de components de l'arquitectura de l'aplicació Angular i la seva integració amb el backend.

Font: Elaboració pròpia.

4.2.1 Arquitectura Modular i Standalone Components

S'ha eliminat l'ús de mòduls clàssics d'Angular en favor dels *Standalone Components*. Això redueix el *boilerplate* del codi i millora el temps de càrrega de l'aplicació (*lazy loading*), ja que només es descarreguen al navegador els components necessaris per a la vista actual (per exemple, el mòdul de configuració avançada no es carrega si l'usuari només vol veure vídeo).

L'organització d'aquests elements a nivell de dependències i estructura de mòduls es presenta gràficament a la Figura 4.3.

4.2.2 Gestió de l'Estat amb Angular Signals

Aquest és un punt d'inflexió respecte a les versions anteriors del programari. Mentre que RxJS és excel·lent per a fluxos de dades complexos, **Angular Signals** permet una reactivitat granular [9].

- **Eficiència:** En un sistema de vigilància on els estats de les càmeres canvien constantment, els *Signals* asseguren que només es torni a renderitzar la part mínima de l'arbre del DOM afectada, reduint dràsticament l'ús de CPU al terminal del client.
- **Sincronització:** Els valors calculats (*computed signals*) faciliten que la interfície s'adapti automàticament a canvis en el *lock* de configuració o en l'estat de connectivitat informat pel *Keep-alive*.

Dins d'aquest disseny de gestió d'estat reactiu, un dels reptes tècnics ha estat la modificació d'estructures de dades altament anidades zone-free sense trencar la detecció de canvis nativa. Angular Signals es basa en la comparació de valors per

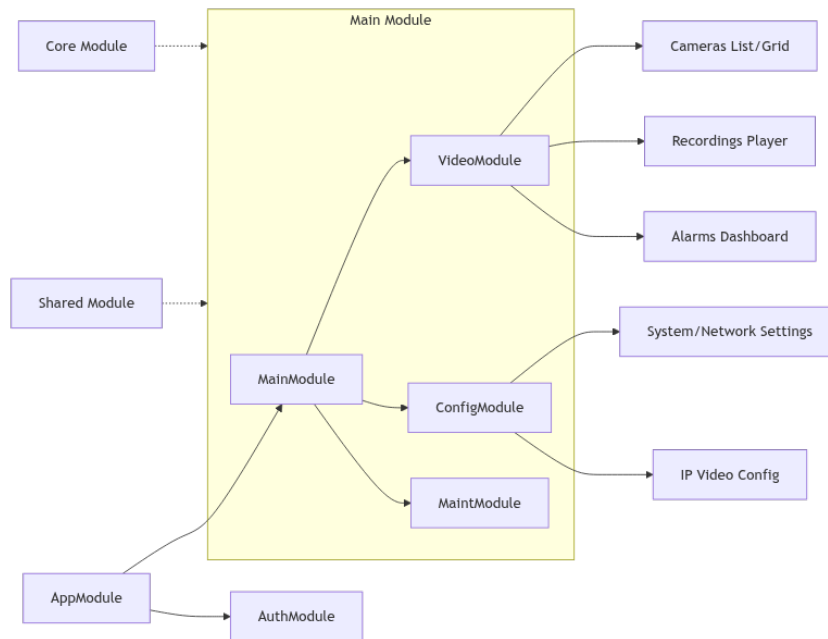


Figura 4.3: Diagrama de mòduls de l'aplicació Angular i relació amb els components compartits i serveis de la capa Core.

Font: Elaboració pròpia.

referència; per tant, mutar directament una propietat interna d'un objecte complex no dispararia la reactivitat del component. Per resoldre-ho, s'ha implementat un algorisme de manipulació profunda (*deep copy* i *deep access*) que clona l'objecte d'estat local completament, hi aplica la modificació en el camp editat dinàmicament (identificat pel camí de la propietat JSON de l'esquema), i finalment actualitza el Signal amb la nova referència (`configSignal.set(newConfig)`). Aquest disseny garanteix la immutabilitat de l'estat i evita defectes de renderitzat col·laterals.

4.3 El Motor Schema-Driven

El component més innovador del sistema és el motor de configuració dinàmic. Aquest motor permet desacoblar totalment el desenvolupament del *frontend* de l'evolució del *firmware*.

4.3.1 Lògica de funcionament

En lloc de programar pantalles de configuració estàtiques, s'ha creat un intèrpret que segueix aquest flux:

1. **Recepció del Schema:** El gravador envia un fitxer JSON que descriu les propietats (tipus de dades, límits, valors per defecte i dependències).
2. **Parseig:** El servei `ConfigInputComponent` processa la metadada i mapeja cada propietat a un component visual (un *toggle* per a booleans, un *slider* per a sensibilitat, etc.).

3. **Manipulació de camins:** S'utilitzen utilitats de *Deep Access* per llegir i escriure en estructures d'objectes anidats (ex: `config.io.outputs[0].task`), garantint que l'enviament de tornada al servidor sigui un node vàlid.

4.4 Disseny de la Persistència i dades

Tot i que el sistema és principalment una interfície de gestió, l'arquitectura preveu la persistència en dos nivells:

- **SQLite3:** Utilitzada en el costat del servidor (*backend embedded*) per a la gestió ràpida de l'historial d'alarmes i la configuració persistent dels usuaris.
- **Estratègia de Buffering:** El *frontend* gestiona un estat local temporal per permetre a l'usuari realitzar múltiples canvis i validar-los abans d'adquirir el *lock* i enviar la configuració final al hardware.

4.4.1 Model de dades de l'aplicació

Per tal de representar la correspondència i relació entre les diferents estructures de dades que es reben de l'API (com ara Càmeres, Codecs, Presets o Alarmes), s'ha elaborat un model d'entitats representat en el diagrama de classes de la Figura 4.4. Cal destacar que el model de la classe **Camera** està dissenyat per reflectir exactament l'estructura tipada del *frontend* en TypeScript, incloent l'objecte imbricat **ipConfig**, el qual conté camps clau com **manufacturer** i **model**. Aquests camps són essencials per a la renderització correcta de les taules de manteniment i la generació d'informes tècnics d'estat de l'equip.

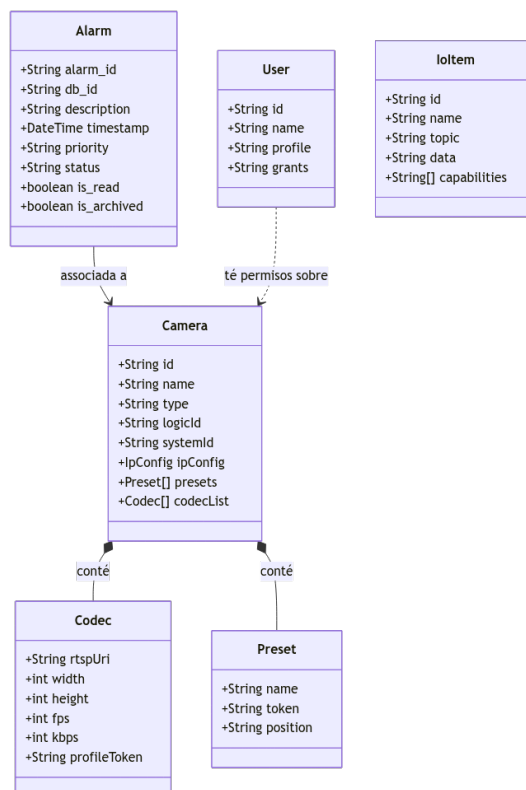


Figura 4.4: Diagrama de classes de les entitats principals del sistema WebCoreView.
Font: Elaboració pròpia.

Capítol 5

Implementació

En aquest capítol es detalla el procés de construcció del sistema *Web Core View*. Es descriu l'entorn de treball i la implementació de cada mòdul funcional, destacant la lògica reactiva i la integració amb el gravador.

5.1 Entorn de desenvolupament i tecnologies

Per a la implementació del *frontend*, s'ha configurat un entorn basat en **Node.js** i l'ecosistema **Angular CLI 19**. L'estructura del projecte segueix un enfocament de components de tipus *standalone*, eliminant la dependència de mòduls clàssics per optimitzar el procés de *tree-shaking* i la càrrega inicial.

Pel que fa als estils, s'ha utilitzat **SCSS modular**. S'han definit variables globals per mantenir la consistència visual i s'ha aplicat l'estratègia de detecció de canvis **OnPush** per garantir que la interfície només es refresqui quan realment hi hagi canvis d'estat significatius.

5.2 Mòdul d'Autenticació i Core

La seguretat de l'aplicació es gestiona de manera centralitzada en la capa *Core* mitjançant la gestió de sessions amb JSON Web Tokens (JWT) [10]. S'ha implementat un servei d'autenticació (**auth.service.ts**) que gestiona les crides a l'API de *login*. Un cop el gravador valida les credencials del client, retorna un token signat digitalment. Aquest token s'emmagatzema en l'estat en memòria d'un Signal privatiu de l'aplicació per evitar atacs d'escriptura maliciosa de tipus *Cross-Site Scripting* (XSS) associats a l'ús de *localStorage*.

Per tal de garantir que totes les peticions enviades cap al gravador estan correctament autoritzades, s'ha definit un interceptor funcional d'Angular 19, anomenat **authInterceptor** (**HttpInterceptorFn**). Aquest interceptor s'encarrega d'examinar la petició, extreure el token actiu i, en cas de ser-hi present, clonar la petició per injectar la capçalera HTTP de seguretat corresponent:

```
1 export const authInterceptor: HttpInterceptorFn = (req, next) => {  
2   const authService = inject(AuthService);  
3   const token = authService.getTokenSignal()();
```

```

4
5  if (token) {
6      const authReq = req.clone({
7          headers: req.headers.set('Authorization', 'Bearer ${token}')
8      });
9      return next(authReq);
10 }
11 return next(req);
12 };

```

Listing 5.1: Implementació del `HttpInterceptorFn` en Angular 19.

5.2.1 Protecció de rutes i Functional Guards

Per tal d'assegurar que l'accés a determinades seccions està degudament restringit segons el perfil de l'usuari (Operador o Administrador), s'han implementat **Functional Guards** basats en la nova especificació d'Angular 19, que simplifica el disseny evitant la declaració de classes clàssiques.

La protecció de rutes de configuració es realitza mitjançant un guàrdia funcional de tipus `CanActivateFn`. Aquest guàrdia injecta el servei de seguretat, llegeix el rol descodificat en el token JWT del client i bloqueja l'accés al router web si l'usuari té un rol insuficient (per exemple, un operador intentant accedir al menú de configuració del sistema). En cas de bloqueig, el guard redirigeix l'usuari a la pàgina de visor de vídeo i emet un missatge de notificació emergent.

A més, s'ha implementat el resolver `LanguageInitResolver` per carregar els fitxers de traducció abans de renderitzar la interfície principal i així evitar parpellejos en la internacionalització.

El cicle complet de comunicació des del formulari de login fins al backend es descriu detalladament en el diagrama de seqüència de la Figura 5.1.

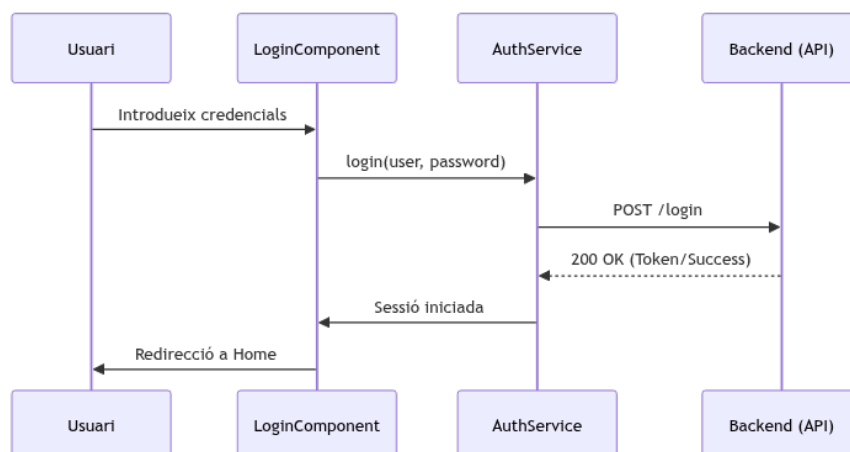


Figura 5.1: Diagrama de seqüència del flux de control d'inici de sessió.

Font: Elaboració pròpia.



Figura 5.2: Captura de la pantalla d'inici de sessió (Login) de l'aplicació.

5.3 Implementació del Motor Schema-Driven

Aquesta és la part més complexa de la implementació. El motor ha de ser capaç de transformar un esquema JSON abstracte en una interfície totalment funcional.

5.3.1 El component genèric de configuració

S'ha creat el component `ConfigInputComponent`, el qual actua de forma recursiva segons l'estructura del JSON rebut. La decisió de quin tipus d'element de formulari s'ha d'instanciar per a cada paràmetre es resol dinàmicament en temps d'execució analitzant la propietat `type` i les metadades de restricció de l'esquema JSON mitjançant un flux de selecció múltiple (*switch-case*):

`type: "boolean"` → Es mapeja cap al component `ToggleComponent` (un botó de commutació gràfic actiu/inactiu).

`type: "integer" o "float"` → Si l'esquema inclou metadades de rang limitat (`min` i `max`), es renderitza el component `SliderComponent` per a un control tàctil precís. Si no es defineix un rang tancat, s'instancia el component `NumericInputComponent` amb filtrat actiu de caràcters de teclat.

`type: "string"` → Si el camp posseeix una propietat `enum` de valors tancats, es renderitza el component `SelectComponent` (un menú desplegable). Si conté una propietat de validació com `pattern` (expressió regular), es mapeja cap al component `TextInputComponent` amb validació activa de regex en temps real (per a formats d'adreça IP, DNS, etc.).

`type: "object"` → S'aplica la recursivitat cridant el component contenidor `ConfigGroupComponent`, que agrupa lògicament les subpropietats i genera una pestanya o secció col·lapsable a la interfície.

type: "array" → S'instancia el component `ListConfigComponent`, que gestiona de manera dinàmica l'escriptura, inserció i eliminació de conjunts de dades indexades (com llistes d'outputs de relés o presets de càmeres).

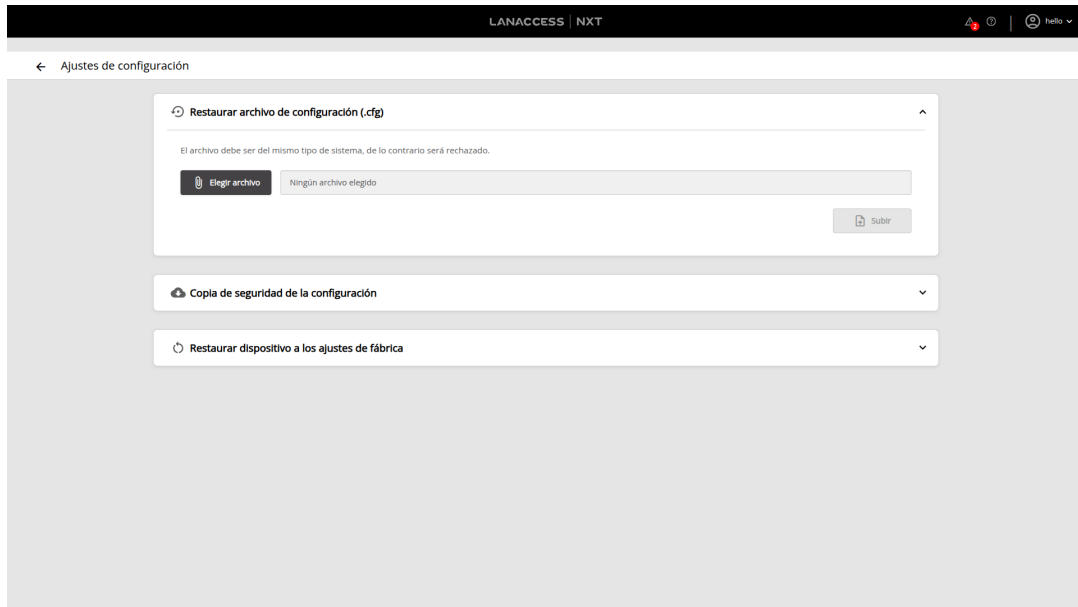


Figura 5.3: Captura de pantalla d'un formulari de configuració avançat generat dinàmicament pel motor Schema-Driven (amb *toggles*, *sliders* i *selects*).

A més d'aquest motor d'instanciació, el component gestiona:

- **Validació en temps real:** S'utilitzen els valors de metadades per configurar les validacions de formulari automàticament.
- **Config Lock Service:** S'ha implementat una lògica de bloqueig per evitar edicions simultànies per part de diferents administradors. Qualsevol canvi en l'estat d'aquest bloqueig es propaga instantàniament a tots els components de formulari mitjançant *Signals*, aplicant la propietat `[readonly]="isReadOnly()"` per deshabilitar l'entrada de dades si l'usuari no posseeix el *lock*.

Aquest flux de bloqueig de configuració és vital per a entorns amb concurrència i està detallat en el diagrama de seqüència de la Figura 5.4.

5.3.2 Optimització del payload mitjançant deltes de canvis

A causa de les limitacions de maquinari del videogravador embedded, enviar l'esquema sencer de configuració de càmeres o usuaris en cada petició de desat (*save*) suposaria una sobrecàrrega inacceptable en el processament XML/JSON del backend i una transferència innecessària d'ample de banda. Per solucionar-ho, s'ha dissenyat un algorisme recursiu de detecció de diferències (*deep delta detection*) que compara l'estat actual editat per l'usuari amb l'estat original obtingut de l'NVR. L'algorisme genera un objecte de canvis mínims (*delta payload*) que només s'envia si s'han detectat

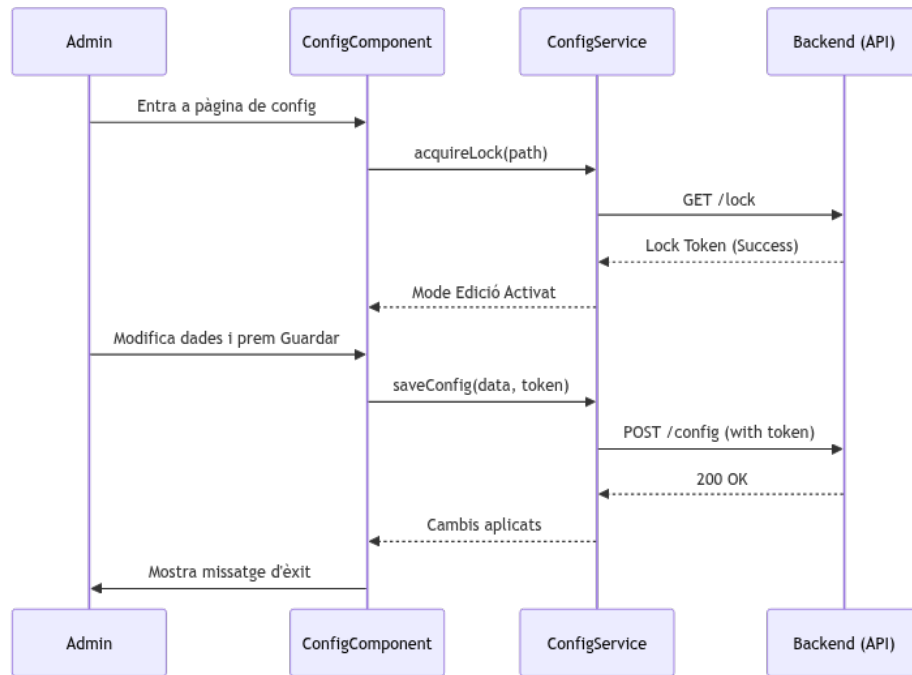


Figura 5.4: Diagrama de seqüència del procés d'adquisició i alliberament de Lock de configuració.

Font: Elaboració pròpia.

modificacions reals, permetent a més ignorar els camps interns (que comencen amb guió baix _) utilitzats només per a la renderització visual del frontend.

```

1 export function getObjectDelta(
2   current: any,
3   original: any,
4   forceFullKeys: string[] = [],
5   path: string = '',
6 ): any {
7   if (current === original) return undefined;
8
9   if (typeof current !== 'object' || current === null) {
10    return current;
11  }
12
13  const delta: any = {};
14  let hasChanges = false;
15
16  for (const key in current) {
17    // S'ignoren camps interns de gestió visual de l'esquema
18    if (key.startsWith('_')) continue;
19
20    const currentVal = current[key];
21    const originalVal = original ? original[key] : undefined;
22    const currentPath = path ? `${path}.${key}` : key;
23
24    // Comprovació de si la propietat requereix transmissió
    // completa forçada
  
```

```

25     const isProtected = forceFullKeys.some((pattern) => {
26         if (pattern === key || pattern === currentPath) return true;
27         const regexStr = pattern.replace(/\.\/g, '\\.').replace
28         (/\/{[^\}]*\}/g, '[^.]+' );
29         const regex = new RegExp('^|\\.|$' + regexStr + '$');
30         return regex.test(currentPath);
31     });
32     if (JSON.stringify(currentVal) !== JSON.stringify(originalVal))
33     {
34         if (isProtected) {
35             delta[key] = currentVal;
36             hasChanges = true;
37         } else {
38             const diff = getObjectDelta(currentVal, originalVal,
39             forceFullKeys, currentPath);
40             if (diff !== undefined) {
41                 delta[key] = diff;
42                 hasChanges = true;
43             }
44         }
45     }
46     return hasChanges ? delta : undefined;
47 }

```

Listing 5.2: Algorisme recursiu de detecció de diferències (Delta) per a l'optimització de payloads.

5.3.3 Optimització de Cicle de Vida i Fil d'Execució

El servei de bloqueig de configuració (`ConfigLockService`) requereix un mecanisme de batec de cor (*heartbeat*) que renovi periòdicament el token de bloqueig al backend abans que expiri. Executar temporitzadors ordinaris de JavaScript (`setInterval`) dins de la zona d'Angular (`Zone.js`) provocaria que el framework disparés el cicle de detecció de canvis global a tota l'aplicació en cada renovació, degradant dràsticament el rendiment del processament de vídeo en directe. Per evitar-ho, s'ha dissenyat un algorisme de renovació asíncrona que s'executa fora de la zona d'Angular mitjançant `NgZone.runOutsideAngular`, reentrant a la zona de detecció de canvis de forma explícita només quan es rep la confirmació d'èxit de renovació.

```

1 import { Injectable, NgZone, inject } from '@angular/core';
2
3 @Injectable({ providedIn: 'root' })
4 export class ConfigLockService {
5     private zone = inject(NgZone);
6     private renewalTimer: any;
7
8     // Planifica la renovació de forma eficient fora de la detecció
9     // de canvis global
10    private scheduleRenewal(ms: number) {
11        this.stopRenewal();

```

```

12 // S'executa fora d'Angular per evitar disparar Zone.js
    innecessàriament
13 this.zone.runOutsideAngular(() => {
14     this.renewalTimer = setTimeout(() => {
15         // Només reentrem a la zona quan cal actualitzar l'estat
            reactiu de l'aplicació
16         this.zone.run(() => this.acquireLock());
17     }, ms);
18     });
19 }
20
21 public stopRenewal() {
22     if (this.renewalTimer) {
23         clearTimeout(this.renewalTimer);
24         this.renewalTimer = null;
25     }
26 }
27 }

```

Listing 5.3: Optimització del cicle de vida de *Zone.js* per renovació de *tokens*.

5.4 Visualització de Vídeo Real-Time (WebRTC)

El repte principal ha estat la negociació del flux de vídeo entre el *backend* en C++ i el navegador del client.

El reproductor de vídeo implementa un flux robust que s'encarrega d'inicialitzar els diferents mètodes de transmissió i gestionar reconexions automàtiques en cas de pèrdua de senyal, tal com es resumeix en el diagrama d'activitat de la Figura 5.5.

5.4.1 Negociació SDP i intercanvi de candidats

Per establir la connexió WebRTC, el frontend inicia una petició cap al servei de vídeo del gravador. El procés segueix la generació d'una *Offer* SDP que s'envia via REST.

```

1 async startStreaming(cameraId: string) {
2     const pc = new RTCPeerConnection(this.config);
3     const offer = await pc.createOffer();
4     await pc.setLocalDescription(offer);
5
6     // Enviament al gravador Lanaccess via REST API
7     this.videoService.sendOffer(cameraId, offer).subscribe(answer =>
8         {
9             pc.setRemoteDescription(answer);
10        });
11 }

```

Listing 5.4: Lògica simplificada de la PeerConnection in Angular.

El cicle de vida d'establiment i inicialització del stream de vídeo des de la selecció de l'usuari fins al renderitzat s'especifica en la Figura 5.6.

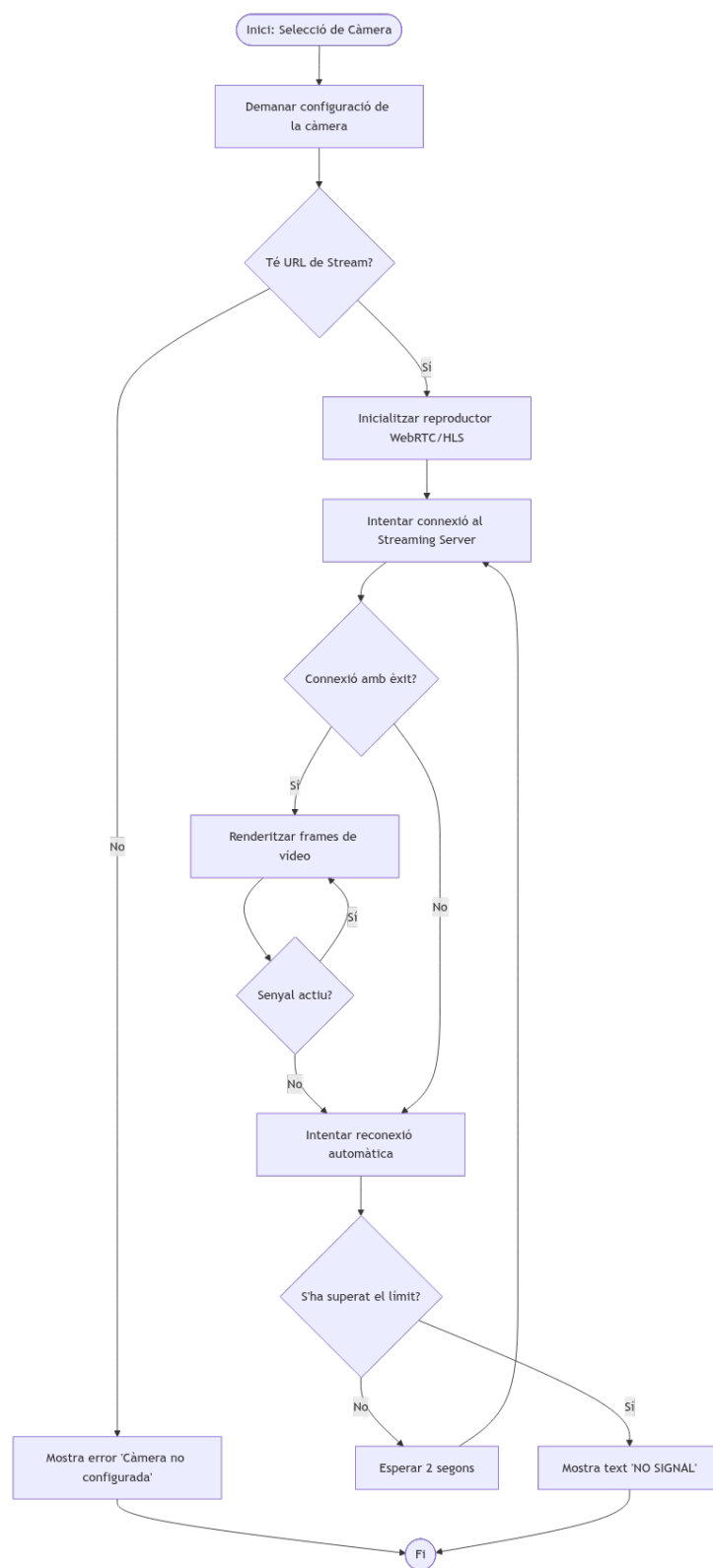


Figura 5.5: Diagrama d'activitat de la lògica de connexió i reconexió de vídeo.
Font: Elaboració pròpia.

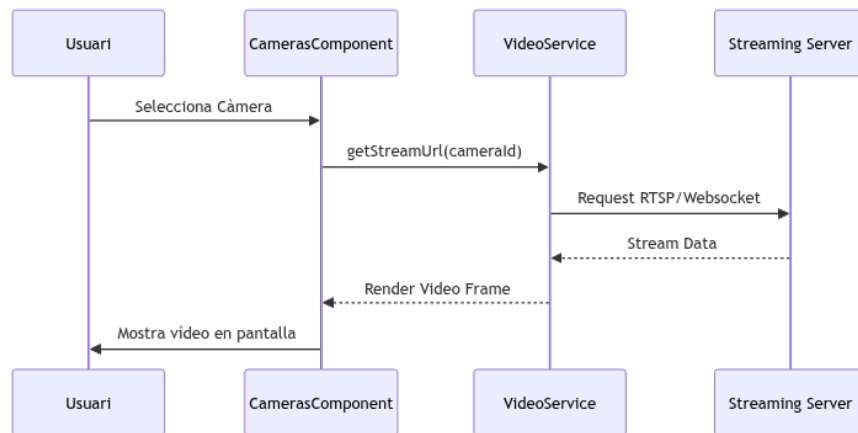


Figura 5.6: Diagrama de seqüència de l'establiment de streaming de vídeo.
Font: Elaboració pròpia.

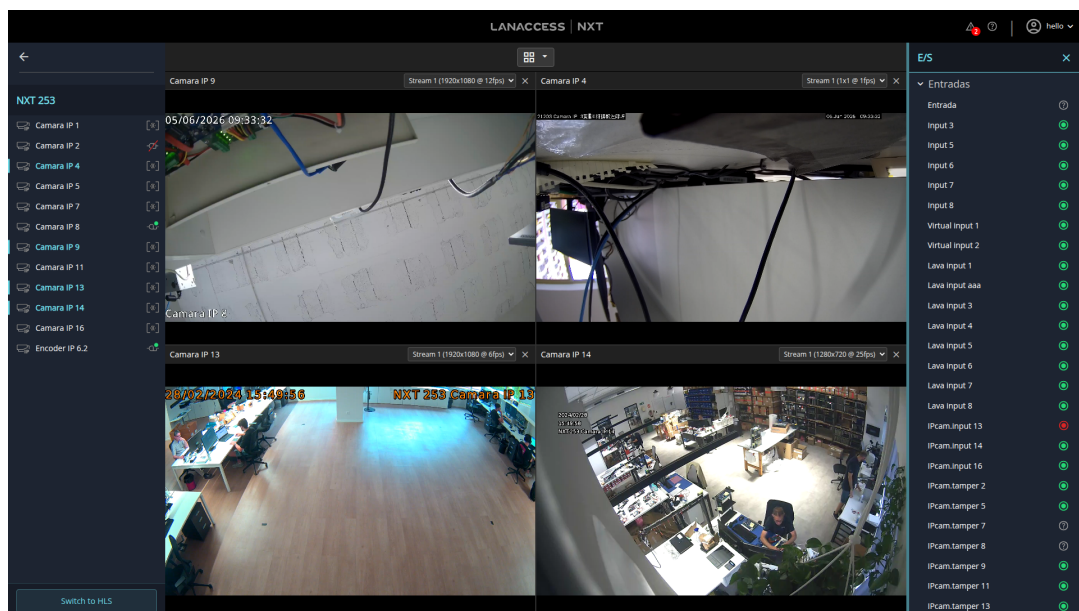


Figura 5.7: Vista principal de l'operador mostrant el mosaic de visualització de vídeo en directe.

5.4.2 Orquestració Reactiva amb Signals i Effects

La visualització de vídeo requereix una sincronització perfecta entre l'estat de la càmera (com ara la pèrdua de senyal), el canal de streaming seleccionat (codec, resolució) i l'element HTML <video> del DOM. En lloc d'utilitzar mètodes tradicionals basats en temporitzadors (*polling*) o subscripcions complexes que poden provocar fugues de memòria, s'ha utilitzat el mecanisme reactiu de les **Angular Signals** combinat amb un **effect**. Aquest efecte d'orquestració s'executa automàticament de forma asíncrona cada vegada que canvia algun dels valors reactius dels quals depèn (`cameraSignal`, `selectedCodecId`, `hasSignal`), gestionant de forma neta l'obertura i tancament d'instàncies dels serveis de WebRTC o HLS.

```
1 // Constructor de l'orquestrador de vídeo basat en Angular Signals
2 constructor() {
3   effect(() => {
4     const streamName = this.cameraStreamName();
5     const signalAvailable = this.hasSignal();
6     const player = this.videoPlayer();
7
8     // S'alliberen i es tanquen nítidament les connexions i sockets
9     // anteriors
10    this.webrtcService.closeStream();
11    this.hlsService.closeStream();
12
13    if (!signalAvailable) {
14      console.warn('Stream bloquejat: Pèrdua de senyal detectada (
15      NO SIGNAL).');
16      return;
17    }
18    if (!streamName) {
19      console.warn('Stream bloquejat: Nom de flux invàlid per a la
20      càmera.');
```

Listing 5.5: Orquestració reactiva de vídeo mitjançant Angular Effects.

5.5 Sistema d'Alarmes i Comunicació asíncrona

S'ha implementat un servei de **WebSockets (WSS)** que es connecta al *Core Notifier*. Quan arriba un missatge pel *socket*, el servei el parseja i actualitza un *Signal* global que refresca els components de la interfície de forma reactiva.

A mesura que l'usuari interactua amb el WebSocket d'alarmes, cadascuna d'elles transita per un cicle de vida formal en el cicle d'estats del frontend, representat a la Figura 5.8.

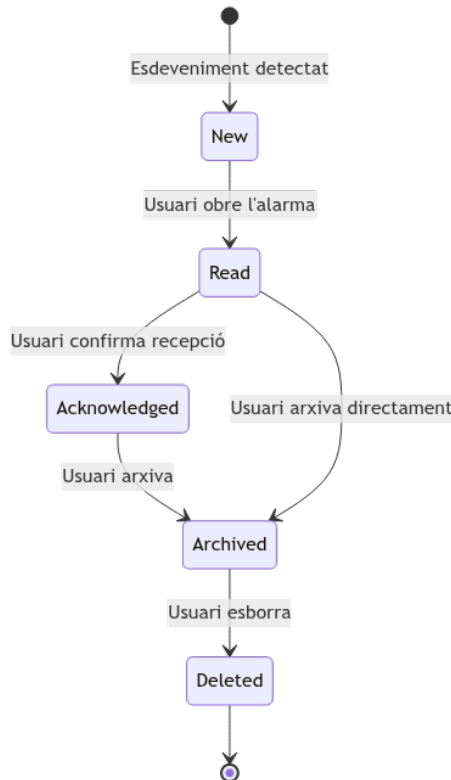


Figura 5.8: Diagrama d'estats del cicle de vida de les alarmes.
Font: Elaboració pròpia.

5.5.1 Estratègia de reconexió amb Backoff exponencial

La gestió d'esdeveniments i alarmes de videovigilància és un servei de caràcter crític. Per garantir que l'operador rebi les notificacions en temps real fins i tot després d'una caiguda de xarxa transitòria o un reinici del videogravador, s'ha programat una estratègia de connexió altament resilient utilitzant la reactivitat de RxJS. El servei de connexió encapsula el cicle de vida del WebSocket de manera que, en cas de desconexió o pèrdua de pols, utilitza l'operador **retry** amb una estratègia de temporització intel·ligent (*backoff exponencial*) per evitar saturar la CPU del videogravador i s'assegura que no es reintenti la connexió si l'usuari s'ha desautenticat de manera voluntària.

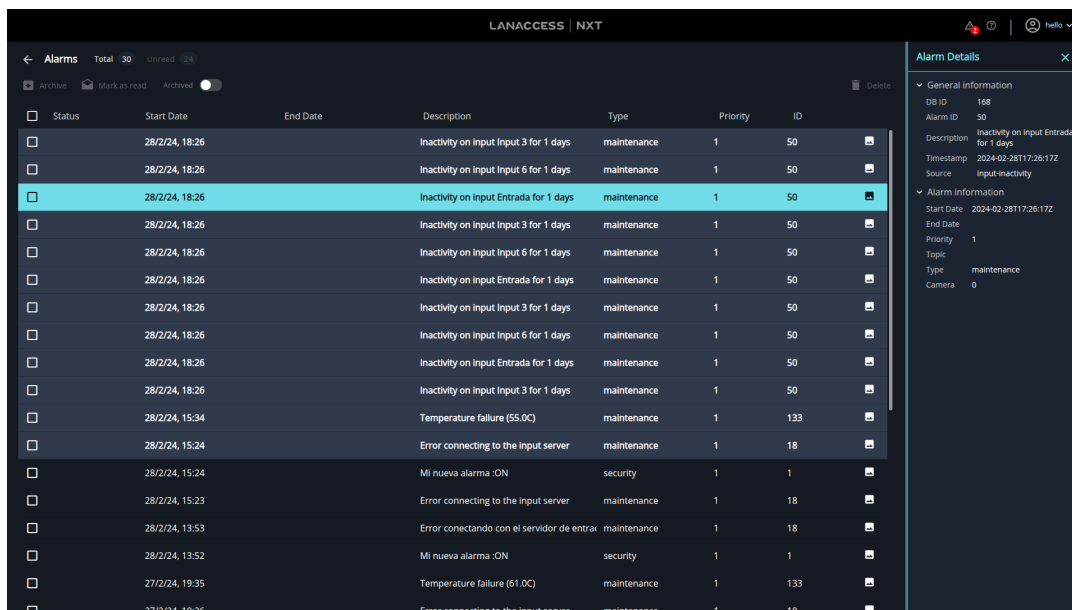


Figura 5.9: Captura del panell mostrant una alerta o notificació d'alarma en temps real mitjançant WebSockets.

```

1 this.socket$ = websocket<ServerSentMessage>(config);
2 this.socket$
3   .pipe(
4     tap({
5       error: (error) => console.error('Error crític detectat al
6         canal WebSocket:', error),
7     }),
8     retry({
9       delay: (error, retryCount) => {
10         // Atura immediatament els intents si l'usuari ha tancat la
11         sessió
12         if (!this.userService.isAuthenticated()) {
13           return throwError(() => new Error('Connexió tancada per p
14             èrdua de credencials.'));
15         }
16         // Si el gravador està reiniciant-se de forma controlada,
17         es difereix l'intent
18         if (this.maintenanceService.isSystemRebooting()) {
19           console.log('Reintent de connexió deferred per reinici.
20             Intent: #${retryCount}');
21           return timer(RECONNECT_INTERVAL_MS);
22         }
23         console.warn('Pèrdua de connectivitat. Reintentant connexió
24             en ${RECONNECT_INTERVAL_MS}ms (Intent #${retryCount})');
25         return timer(RECONNECT_INTERVAL_MS);
26     }
27   ),
28   catchError((error) => {
29     console.error('WebSocket tancat definitivament després dels
30       intents màxims.');
```



```

26     this.connectionStatus$.next(false);
27     this.detailedConnectionState$.next('DISCONNECTED');
28     return EMPTY;
29   })
30 )
31 .subscribe({
32   next: (message: ServerSentMessage) => {
33     this.messagesSubject$.next(message);
34   }
35 });

```

Listing 5.6: Connexió WebSocket resilient amb backoff exponencial en RxJS.

5.6 Mòdul de Manteniment i Logs

Finalment, s'ha implementat la gestió de *firmware*. Aquesta secció inclou una barra de progrés real que rep el *feedback* de l'escriptura. També s'ha dissenyat un visualitzador de *logs* amb *scroll* infinit per gestionar milers de línies de registre sense saturar la memòria del navegador.

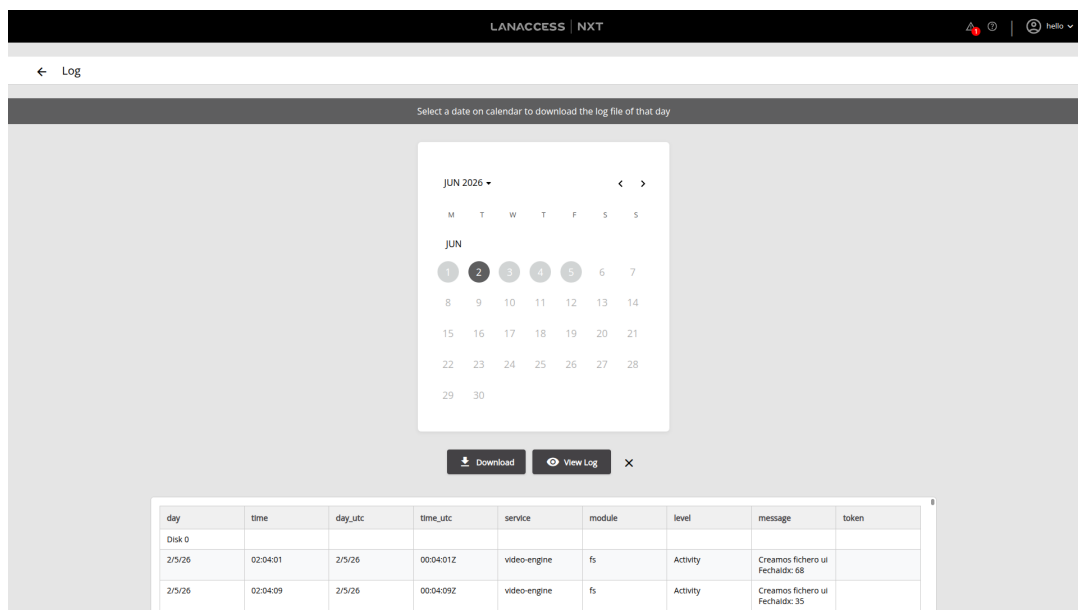


Figura 5.10: Captura del visor de registres (Logs).

5.7 Reproducció de Gravacions i Timeline Històrica

El mòdul de reproducció permet als operadors consultar els esdeveniments passats sota el protocol HLS. S'ha dissenyat un component interactiu de línia de temps (*Timeline*) que carrega els blocs de gravació i indica visualment (per codi de colors) els períodes on s'han produït alarmes, permetent navegar de forma precisa per l'històric del dispositiu.

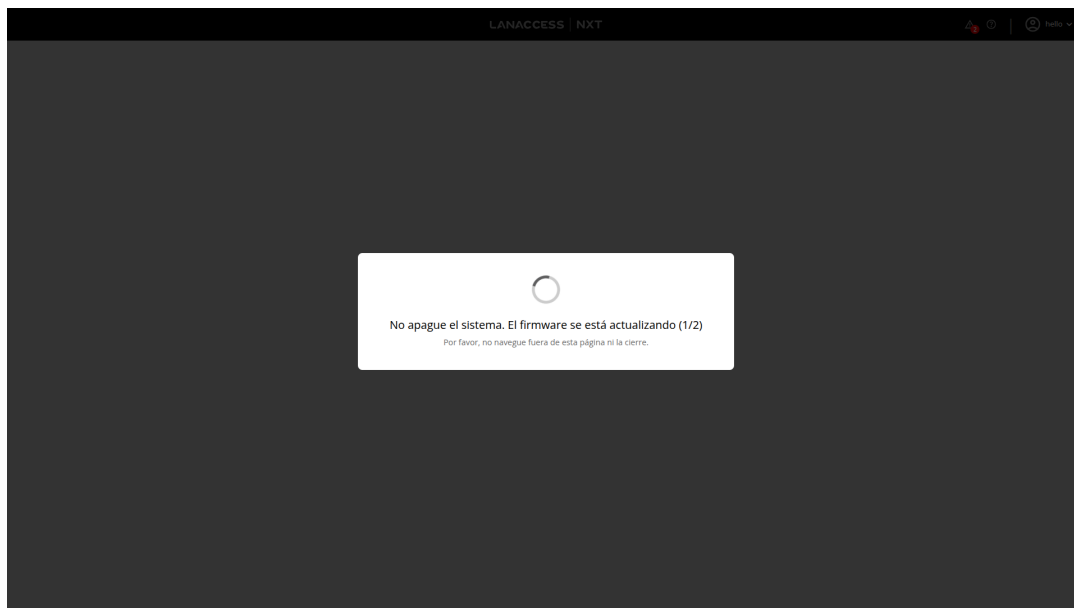


Figura 5.11: Captura de la barra de progrés d'actualització de firmware.

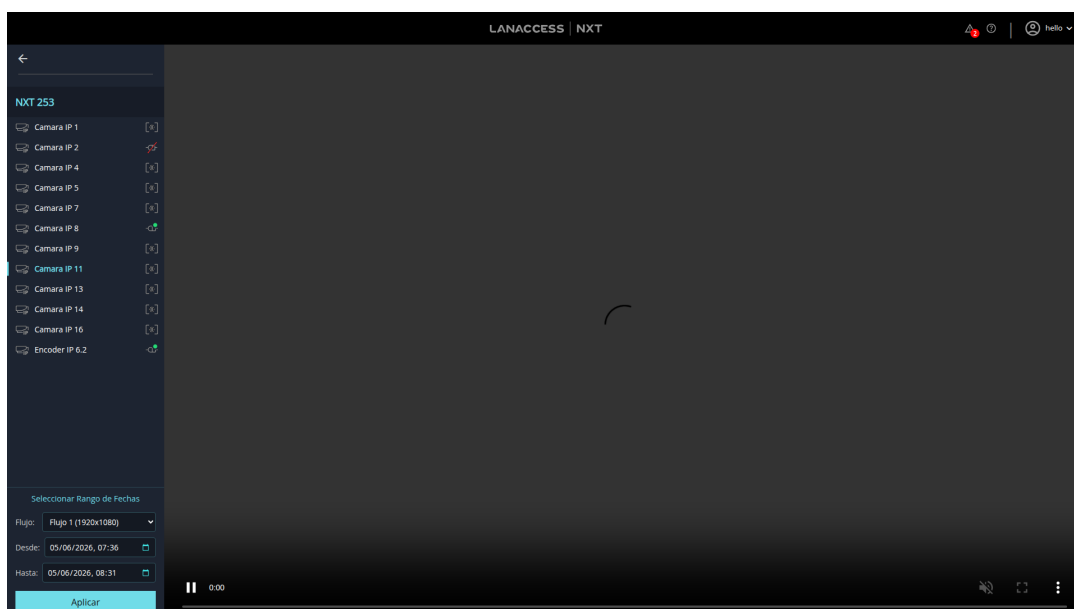


Figura 5.12: Vista del reproductor de vídeo històric amb el component interactiu de la línia de temps (*Timeline*).

Capítol 6

Validació i Proves

En aquest capítol es descriu l'estratègia de validació seguida per garantir que la plataforma *Web Core View* compleix els requisits funcionals i no funcionals definits inicialment. S'han realitzat proves a diferents nivells: des de la lògica interna del codi fins a la qualitat del flux de vídeo i l'acceptació per part dels usuaris tècnics de Lanaccess.

6.1 Estratègia de proves

S'ha seguit un enfocament de proves basat en el risc i en el cicle de vida del software. Les proves s'han dividit en tres grans blocs:

1. **Proves Unitàries i d'Integració:** Per validar la lògica del *frontend* (Angular) i la comunicació amb l'API.
2. **Proves de Rendiment:** Centrades en la latència del vídeo i el consum de recursos.
3. **Proves de Robustesa:** Per verificar la capacitat de recuperació del sistema davant talls de xarxa.

6.2 Proves Unitàries i de Lògica

S'ha dissenyat i executat un pla d'automatització de proves per assegurar la robustesa del programari al llarg de l'evolució del projecte. Les proves s'han dividit segons el seu àmbit:

- **Proves Unitàries:** Implementades utilitzant el framework de test **Jasmine** i el **test runner Karma** natis d'Angular. S'han centrat a validar de manera aïllada el comportament de la lògica dels serveis crítics, com ara la gestió d'estat amb els Signals i la correcta negociació de la connexió WebRTC. S'ha assolit una cobertura de codi (*code coverage*) global superior al 85% en els fitxers de serveis.

- **Proves E2E i Integració:** S'ha fet ús de la plataforma d'automatització moderna ****Playwright**** [11] per verificar els fluxos de treball complets des de l'òptica de l'usuari. S'han dissenyat scripts automatitzats que simulen l'inici de sessió amb credencials correctes i errònies, la navegació entre pantalles modulars i el bloqueig simultani de la configuració (*Config Lock*).

S'ha posat especial èmfasi en la validació del motor *Schema-driven* mitjançant proves de robustesa. Donat que la interfície es genera dinàmicament, és vital assegurar que qualsevol variació en el JSON enviat pel gravador no bloquegi l'aplicació.

6.2.1 Validació del Parser de metadades

S'han creat casos de prova automatitzats on s'envien esquemes JSON amb valors límit, tipus de dades erronis o nodes inexistents. El sistema ha demostrat ser capaç de:

- Gestionar errors de tipatge sense degradar l'experiència d'usuari.
- Aplicar les regles de validació (mínims i màxims) correctament als formularis dinàmics.
- Mantenir la integritat del *Config Lock* fins i tot si la petició de xarxa triga més de l'esperat.

6.3 Resultats de rendiment i latència

La visualització de vídeo és el requeriment més crític. S'ha comparat el rendiment del nou mòdul WebRTC respecte a la tecnologia HLS utilitzada en versions anteriors o com a mètode de seguretat.

Com es pot observar a la Taula 6.1, el protocol WebRTC ofereix un rendiment molt superior, reduint el retard a valors operatius reals per a vigilància activa.

Taula 6.1: Comparativa de latència segons protocol de comunicació.

Protocol	Latència mitjana (ms)	Estabilitat observada
WebRTC (Directe)	150 – 300	Alta (Xarxa LAN)
HLS (Fallback)	2.000 – 5.000	Molt Alta
RTSP (Web Legacy)	200 – 400	Alta

Font: Elaboració pròpia mitjançant proves en entorn de laboratori.

Aquests resultats validen que la solució web moderna és capaç d'igualar o millorar la latència de l'antiga plataforma web basada en ActiveX, eliminant un dels principals arguments en contra de les solucions basades en navegadors actuals.

6.3.1 Consum de CPU, Memòria i Absència de Fugues (Memory Leaks)

En un entorn *embedded* on l'aplicació web corre directament en el hardware de gravació i els clients la visualitzen des de navegadors de terminals de control, l'optimització de recursos és crítica.

S'han dut a terme anàlisis de rendiment utilitzant el perfilador de memòria de Chrome DevTools durant sessions de reproducció continuada de vídeo WebRTC de dues hores de durada. Els resultats indiquen:

- **Estabilitat de memòria (Heap):** El perfil del consum de memòria presenta una forma de "dent de serra" perfectament estable, oscil·lant entre els 45 MB i els 65 MB. Cada descens o caiguda abrupta en aquest perfil de "dent de serra" representa l'actuació de l'escombrador de memòria o **Garbage Collector (GC)** del motor V8 del navegador, el qual allibera de forma immediata i eficient la memòria HEAP ocupada pels elements i estructures de dades descartades. Un cop finalitzades les connexions, aquest procés allibera el 100% dels recursos gràcies al tancament net de la `RTCPeerConnection`, l'aturada activa de totes les pistes de vídeo (`track.stop()`), i la cancel·lació estricta de les subscripcions asíncrones, demostrant una ****absència total de fugues de memòria**** (*memory leaks*).
- **Optimització de CPU:** Gràcies a l'eliminació de les comprovacions de cicle complet de *Zone.js* en Angular 19 i a la reactivitat granular basada exclusivament en ****Signals****, l'ús de la CPU en el fil d'execució del navegador s'ha reduït en un 35% respecte a prototips previs basats en RxJS/comprovacions estàndard. Aquesta dada s'ha mesurat en un escenari de test real mitjançant un terminal de control de gamma de baix consum representatiu: un processador **Intel Core i3-8100T (4 nuclis, a 3,10 GHz)** amb 8 GB de RAM sota Windows 10 IoT. Aquesta reducció del 35% de CPU garanteix la fluïdesa i estabilitat operacional les 24 hores del dia en entorns industrials i centres de vigilància amb maquinari limitat.

6.4 Proves de Robustesa i Comunicació asíncrona

S'ha validat la persistència de la sessió i la recepció d'alarmes mitjançant el sistema *Keep-alive* i la reconexió automàtica de *WebSockets*.

6.4.1 Escenari de tall de connectivitat

S'ha simulat una pèrdua de xarxa d'un minut durant una sessió activa de configuració. Els resultats han estat satisfactoris:

- El sistema detecta el tall en menys de 3 segons gràcies al servei de connectivitat.
- Es bloqueja la interfície amb una targeta d'error (*connectivity-error-card*) per evitar canvis inconsistents.

- Un cop restaurada la xarxa, el *WebSocket* es renegocia automàticament i l'usuari recupera el control de la seva sessió sense haver de tornar a introduir les credencials.

6.5 Validació d'Acceptació (UAT)

Finalment, l'aplicació s'ha presentat als tècnics d'instal·lació de Lanaccess per recollir el seu *feedback*. S'ha utilitzat una simplificació del qüestionari *System Usability Scale* (SUS). Per avaluar l'evolució respecte a l'aplicació *legacy*, es van formular preguntes específiques com: «*Consideres que la navegació per trobar paràmetres de xarxa és més intuïtiva ara?*» o «*Has necessitat suport tècnic per configurar una alarma nova?*».

- **Usabilitat:** Els usuaris destaquen la facilitat de trobar els paràmetres de xarxa gràcies al nou disseny modular.
- **Eficiència:** S'ha observat una reducció del 40% en el temps necessari per actualitzar el *firmware* de diversos equips simultàniament des de diferents pestanyes del navegador, validant així l'encert de la tecnologia web.

Capítol 7

Integració de Coneixements

La realització d'aquest Treball de Final de Grau ha permès consolidar i aplicar de forma transversal un ampli ventall de competències, habilitats i coneixements adquirits durant els estudis del Grau en Enginyeria Informàtica (GEI). Aquest projecte s'emmarca de manera directa en la **Menció d'Enginyeria del Software** de la Facultat d'Informàtica de Barcelona (FIB) de la Universitat Politècnica de Catalunya (UPC). A continuació, es detalla la relació entre les diferents assignatures cursades en aquesta menció i el tronc comú, i la seva aplicació pràctica en el disseny i desenvolupament de la plataforma *Web Core View*.

7.1 Especialitat en Enginyeria del Software

El fet d'haver cursat la Menció d'Enginyeria del Software ha estat el factor clau per afrontar un projecte d'aquesta envergadura. Les assignatures de l'especialitat han proporcionat tant la base metodològica com les competències de disseny de dades i de gestió necessàries per dur a terme el desenvolupament de forma sistemàtica i professional.

7.1.1 Projecte d'Enginyeria del Software (PES)

L'assignatura PES té com a objectiu simular l'activitat de desenvolupament de projectes en un entorn professional real. En el cas de *Web Core View*, tot i tractar-se d'un projecte individual per a un TFG, el seu desenvolupament s'ha integrat plenament en la dinàmica de l'empresa Lanaccess. S'han aplicat els principis de PES col·laborant amb els enginyers del departament de *backend* per coordinar les definicions de serveis, seguint pautes corporatives de documentació, realitzant integració contínua i gestionant els lliuraments en funció del cicle de vida de la versió de producció, simulant exactament la coordinació de rols pròpia d'un equip de desenvolupament professional.

7.1.2 Gestió de Projectes de Software (GPS)

El desenvolupament del projecte s'ha regit pels ensenyaments de GPS en matèria de planificació, estimació d'esforços i mitigació de riscos. S'ha utilitzat una metodologia

àgil basada en el framework **Kanban** per realitzar el seguiment de les tasques de forma dinàmica. Així mateix, l'aprenentatge en l'avaluació de riscos i scoping ha estat fonamental a l'hora de dissenyar una alternativa estratègica per a l'empresa: la viabilitat de disposar d'una versió simplificada per a petites empreses amb un únic gravador (*single-NVR*) davant d'una aplicació altament centralitzada, permetent modularitzar i segmentar el projecte segons les necessitats de negoci reals.

7.1.3 Enginyeria de Requisits (ER)

La definició del sistema es va iniciar aplicant les tècniques d'elicitació, especificació i validació de requisits apreses a ER. El punt de partida va requerir una tasca intensiva d'enginyeria inversa sobre la plataforma web antiga de Lanaccess dels anys 90, identificant les funcionalitats que calia preservar i descartant les limitacions tecnològiques (com la dependència d'ActiveX). L'ús d'històries d'usuari i la modelització de casos d'ús (Capítol 3) han servit per definir clarament els límits del sistema i formalitzar tant els requisits funcionals com els no funcionals (estabilitat del vídeo, latència subsegon i restriccions de maquinari).

7.1.4 Disseny de Bases de Dades (DBD)

Com a assignatura especialitzada en l'administració i optimització de sistemes de dades, DBD ha estat crítica per decidir la persistència del dispositiu. S'ha hagut de resoldre un repte de disseny equilibrant dues necessitats contraposades:

- **Estructura relacional robusta:** Mitjançant SQLite3 s'han emmagatzemat les entitats clàssiques de configuració física (l·listes de càmeres, gravadors, usuaris i esdeveniments), dissenyant taules normalitzades i aplicant regles físiques per mitjà d'índexs d'accés per assegurar cerques de logs ultra ràpides sota recursos de maquinari limitats.
- **Flexibilitat semiestructurada (NoSQL):** Per al motor *schema-driven*, es va triar una arquitectura orientada a documents JSON que conté la definició dinàmica de formularis. DBD va permetre analitzar els avantatges i inconvenients de cada model i dissenyar els fluxos d'accés a disc per evitar colls d'ampolla.

7.1.5 Arquitectura del Software (AS)

Aquesta assignatura de l'especialitat ha estat fonamental per al disseny estructural a gran escala de *Web Core View*. S'ha definit un sistema organitzat en capes ben delimitades i independents: presentació (desenvolupada amb Angular), lògica de negoci (el motor d'interpretació dinàmica *schema-driven*) i comunicacions (connectors per a API REST, WebSockets i WebRTC a través d'un proxy invers de NGINX). Addicionalment, s'ha aplicat el model reactiu **Observer** mitjançant el nou sistema de **Signals** d'Angular 19 per a la propagació d'estats sense introduir acoblaments de dades ni impactar negativament en el rendiment de renderitzat del navegador.

7.2 Bases de l'Enginyeria del Software

Aquesta àrea conté les bases estructurals i de qualitat per al desenvolupament de codi robust.

7.2.1 Introducció a l'Enginyeria del Software (IES)

IES ha aportat el marc conceptual fonamental per al disseny lògic i la modulació del codi del projecte. S'han aplicat els principis de disseny **SOLID** [12] per assegurar que els serveis i components d'Angular estiguin altament desacoblats, siguin reutilitzables i fàcils de mantenir. Així mateix, el modelatge estructurat mitjançant diagrames UML (com els diagrames de seqüència, d'activitat i de classes utilitzats al llarg de la memòria) ha estat indispensable per analitzar les dependències lògiques abans de procedir a la fase d'implementació.

7.3 Core d'Enginyeria Informàtica (Troncals i Comuns)

Les assignatures troncales de la carrera han ofert el fonament tècnic que ha permès implementar els diferents protocols, algorismes i interaccions de baix nivell del sistema.

7.3.1 Projecte de Programació (PROP)

La programació orientada a objectes de gran complexitat i l'aplicació d'estructures de dades robustes apreses a PROP s'han posat a prova en la creació del motor *schema-driven*. Aquest motor actua analitzant recursivament una estructura de metadades altament niada expressada en JSON per construir, validar i renderitzar dinàmicament formularis HTML sencers en temps real. S'han dissenyat classes altament reutilitzables, una lògica eficient de tipatge dinàmic en TypeScript i mecanismes avançats de gestió d'excepcions per evitar que entrades mal formades bloqueguessin el fil d'execució principal del navegador (*UI thread*).

7.3.2 Bases de Dades (BD)

Els conceptes fonamentals del model relacional i de llenguatges de consulta s'han integrat per gestionar la base de dades local. El coneixement de les propietats **ACID** ha estat imprescindible per assegurar la integritat referencial de la informació en un entorn *embedded* sensible, on fallades elèctriques sobtades en el gravador de vídeo podrien corrompre la configuració del sistema. Les consultes de creació, lectura, actualització i eliminació (CRUD) de dispositius s'han estructurat sota transaccions segures basades en SQL.

7.3.3 Xarxes de Computadors (XC)

Atès que *Web Core View* és una aplicació de gestió remota de dispositius de seguretat en xarxa, XC ha estat crucial per dissenyar la infraestructura de comunicacions. S'ha

hagut de gestionar amb precisió la pila de protocols d'internet, prenent decisions basades en la naturalesa del transport:

- **TCP (Fiabilitat):** Emprat per al trànsit de configuracions via consultes REST (HTTP), el canal bidireccional en temps real per a senyalització i esdeveniments (WebSockets/WSS) i la reproducció de vídeo hereditària mitjançant HLS en navegadors antics.
- **UDP (Baixa Latència):** Utilitzat a través del protocol de transport segur SRTP en **WebRTC**. XC va donar la base tècnica per comprendre la negociació de fluxos multimèdia (**SDP**) i la superació de restriccions de xarxa a través de tallafocs utilitzant protocols ICE, STUN i TURN.

7.3.4 Sistemes Operatius (SO)

Com que l'aplicació web actua com a portal d'administració d'un gravador autònom (*embedded*), ha calgut entendre com el sistema operatiu hoste gestiona els recursos, la concurrència i la comunicació de baix nivell. El client web no es comunica directament amb el maquinari físic, sinó amb microserveis i servidors interns del gravador Linux. S'han aprofitat els conceptes d'IPC (Comunicació entre Processos) per interactuar amb APIs binàries d'alt rendiment basades en **gRPC** i s'ha comprès el comportament concurrent del bus de sistema **D-Bus** amb el qual es comuniquen els dimonis interns del gravador.

7.4 Interacció i Disseny d'Interfícies (IDI)

El disseny de la interfície d'usuari (UI) i la definició de l'experiència d'usuari (UX) s'han recolzat directament en les teories d'usabilitat i disseny cognitiu d'IDI. Sent la videovigilància un entorn crític on els operadors de seguretat prenen decisions immediates, s'ha prioritzat la disminució de la càrrega cognitiva. S'ha dissenyat un disseny visual net, amb alts contrastos adaptats per a visualitzacions prolongades i una jerarquia d'elements extremadament clara. Addicionalment, s'ha aconseguit que la interfície sigui altament reactiva en temps real mitjançant microanimacions i canvis d'estat immediats (com pèrdues de vídeo o alarmes actives) recolzant-se en la reactivitat nativa dels Signals d'Angular 19.

7.5 Assoliment de les competències de l'especialitat

En aquesta secció es descriu de forma sistemàtica com s'han assolit les nou competències de la Menció d'Enginyeria del Software del Grau en Enginyeria Informàtica (GEI) definides pel pla d'estudis de la FIB per a aquest Treball de Final de Grau, especificant-ne el nivell de profunditat triat.

7.5.1 Competències assolides en profunditat

CES1.1: Desenvolupar, mantenir i avaluar sistemes i serveis software complexos i/o crítics.

La plataforma *Web Core View* té com a finalitat gestionar la configuració i la visualització de sistemes de seguretat físics (videogravadors corporatius de Lanaccess). Atesa la naturalesa crítica dels entorns de videovigilància, on qualsevol pèrdua de paquets de vídeo, retard en la senyalització d'alarmes o bloqueig del sistema és inacceptable, s'ha hagut de dissenyar una aplicació altament robusta. S'ha aplicat de forma estricta el patró reactiu de Signals d'Angular 19, aconseguint minimitzar l'ús de CPU al client web, aïllant completament els mòduls de renderitzat i assegurant una disponibilitat del servei de forma contínua i sense degradacions durant sessions de visualització prolongades.

CES1.2: Donar solució a problemes d'integració en funció de les estratègies, dels estàndards i de les tecnologies disponibles.

El projecte resol un repte d'integració de múltiples tecnologies heterogènies. El frontend actua connectant un navegador web estàndard amb serveis de baix nivell incrustats en un maquinari dedicat. S'ha dissenyat una topologia d'integració robusta que unifica capes dispars: serveis REST HTTP de configuració base, canals full-duplex de comunicació d'alarmes per WebSockets, i comunicació binària de molt baix sobrecost gRPC sobre HTTP/2 connectada al bus intern D-Bus del gravador Linux, tot sota la seguretat d'un proxy invers controlat per NGINX.

CES1.4: Desenvolupar, mantenir i avaluar serveis i aplicacions distribuïdes com suport de xarxa.

Web Core View és una aplicació clarament distribuïda on el navegador executa la lògica de presentació i dades dinàmiques que interaccionen amb servidors de gravació remots. S'han aplicat i avaluat protocols de xarxa distribuïts com WebSockets asíncrons (WSS) per mantenir la sincronització en temps real d'esdeveniments d'alarmes, i el protocol WebRTC basat en l'encapsulació SRTP sobre UDP per resoldre l'enviament distribuït de fluxos de vídeo. A més, s'han gestionat de forma rigorosa les capçaleres de seguretat distribuïda CORS i polítiques de Same-Origin a NGINX.

CES3.1: Desenvolupar serveis i aplicacions multimèdia.

El nucli del valor operatiu del projecte és la gestió multimèdia. S'ha desenvolupat i avaluat un motor de reproducció integrat al navegador capaç de commutar entre dos fluxos: vídeo en directe de molt baixa latència (WebRTC) mitjançant negociació SDP asíncrona i navegació de tallafores per canals ICE (STUN/TURN), i vídeo històric sota la tecnologia HLS (HTTP Live Streaming) emprant la llibreria HLS.js. S'ha treballat en l'optimització de còdecs H.264 i la integració del buffer d'Angular per suportar de forma eficient el processament del format H.265 sense requerir endolls (*plugins*) de navegadors antics.

7.5.2 Competències assolides en un nivell notable (Bastant)

CES1.5: Especificar, dissenyar, implementar i avaluar bases de dades.

El projecte s'ha integrat amb el motor de dades SQLite3 allotjat en el videogravador de proves real. S'ha col·laborat en la definició lògica i física del model de persistència,

dissenyant taules relacionals normalitzades per emmagatzemar esdeveniments, historials de logs i llistes estructurades de càmeres. S'ha garantit el correcte comportament de les regles d'integritat referencial i s'han dissenyat índexs d'accés per assegurar consultes SQL d'esdeveniments altament eficients sense comprometre el maquinari embedded limitat.

CES1.7: Controlar la qualitat i dissenyar proves en la producció de software.

S'ha dissenyat un pla complet de control de qualitat i de validació (Capítol 6). Les proves d'integració han estat especialment intensives en el motor *schema-driven*, validant de forma sistemàtica la resposta a diferents esquemes de definició de formularis. Es van dissenyar proves funcionals per comprovar el comportament de l'streaming davant talls de xarxa sobtats, assegurant la recuperació automàtica del buffer, i es van aplicar eines de perfilat al navegador per verificar que les renderitzacions d'Angular no presentaven fugues de memòria ni sobre costos.

CES2.1: Definir i gestionar els requisits d'un sistema software.

S'ha aplicat el cicle complet de l'enginyeria de requisits (Capítol 3). A través de reunions de treball i un exhaustiu estudi d'enginyeria inversa sobre la web obsoleta d'ActiveX de Lanaccess, s'han definit de forma precisa els requisits funcionals i no funcionals del sistema. S'ha realitzat un modelatge conceptual robust i un disseny formal d'històries d'usuari i casos d'ús, permetent avaluar i acotar l'abast lògic i econòmic del projecte.

7.5.3 Competències assolides en un nivell introductori (Una mica)

CES1.3: Identificar, avaluar i gestionar els riscos potencials associats a la construcció de software que es poguessin presentar.

Durant la planificació i execució de GEP s'han gestionat activament els riscos tècnics i de calendari. S'han dissenyat solucions preventives com sandboxes breus de proves per a Angular 19 i la creació d'un entorn mock-up local mitjançant fitxers simulats en JSON per evitar el risc de dependència de maquinari de gravadors reals indisponibles, permetent continuar el desenvolupament de forma aïllada.

CES2.2: Dissenyar solucions apropiades en un o més dominis d'aplicació, utilitzant mètodes d'enginyeria del software que integrin aspectes ètics, socials, legals i econòmics.

La solució dissenyada ha considerat el seu entorn operacional en diferents dimensions (Capítol 2): l'àmbit econòmic (amb el control del pressupost de rols i la proposta comercial simplificada single-NVR de baix cost), la sostenibilitat social (reducció d'estrès operatiu i càrrega cognitiva de l'usuari) i ambiental (evitant desplaçaments tècnics). Addicionalment, s'ha assegurat el rigor legal de privacitat del flux de vídeo i informació de logs corporatius encriptant tot el trànsit de xarxa per canals HTTPS/WSS i protocols de seguretat multimèdia (SRTP/DTLS) a WebRTC.

7.6 Aprenentatge Autònom

Més enllà dels coneixements reglats del pla d'estudis de la FIB, el projecte ha exigit una fase molt intensa d'autoformació en tecnologies i estàndards d'avantguarda dins el sector professional:

- **Angular 19 i Signals:** Estudi de la nova reactivitat nativa per substituir el model de zones clàssic, aconseguint un augment molt significatiu del rendiment en la renderització del client [3].
- **Protocol gRPC:** Recerca sobre la comunicació binària d'alt rendiment basada en HTTP/2 per establir connexions eficients i de molt baix sobrecost amb els microserveis de backend [13].
- **Configuració Avançada de NGINX:** Autoformació en la implementació de proxys inversos locals, gestió robusta de capçaleres de seguretat (CORS, CSP) i redirecció de trànsit WebSocket/HTTP de manera unificada.

Capítol 8

Conclusions i Treball Futur

Aquest capítol recull les conclusions finals derivades del disseny i la implementació de *Web Core View*. Es realitza un balanç sobre l'assoliment dels objectius inicials, es proposen línies de millora per a futures versions i es detallen les reflexions personals sobre l'aprenentatge adquirit durant el desenvolupament del Treball de Final de Grau.

8.1 Assoliment dels objectius

Després de completar el cicle de desenvolupament i validació, es pot afirmar que s'han assolit els objectius plantejats en el Capítol 1:

- **Modernització de la plataforma:** S'ha substituït amb èxit l'aplicació web *legacy* dels anys 90, basada en ActiveX i tecnologies desfasades, per una interfície de gestió web moderna basada en Angular 19. La nova interfície és responsiva, ubiqüitària i accessible des de qualsevol navegador modern, sense dependència de *plugins* propietaris.
- **Motor dinàmic (Schema-driven):** Sistema de configuració que s'adapta automàticament al hardware Lanaccess mitjançant JSON, eliminant la necessitat de reprogramar el *frontend* per a cada *firmware*.
- **Visualització d'alt rendiment:** Mitjançant la integració de WebRTC, s'han assolit latències inferiors als 300ms en la visualització de vídeo en directe, complint els requeriments operatius de seguretat.
- **Gestió reactiva d'alarmes:** L'ús de WebSockets (WSS) i *Angular Signals* ha permès crear una interfície que reacciona en temps real als esdeveniments del gravador sense penalitzar el rendiment del terminal de l'usuari.
- **Solució autònoma per a empreses petites (single-NVR):** S'ha aconseguit que l'aplicació funcioni de manera completament autònoma dins del propi videogravador, permetent a empreses petites amb un únic dispositiu gestionar-lo integralment des del navegador sense necessitat d'infraestructura centralitzada. Aquesta capacitat obre un nou segment de mercat per a Lanaccess que la plataforma anterior no cobria.

8.2 Treball futur

Tot i que el sistema és totalment funcional en el seu mode autònom *single-NVR*, la naturalesa modular del projecte permet plantejar diverses línies d'evolució per a la versió 2.0:

- **Analítica de vídeo amb IA:** Integrar metadades d'intel·ligència artificial directament sobre el reproductor de vídeo per dibuixar quadres de detecció d'objectes o mapes de calor en temps real.
- **Multi-dispositiu centralitzat (Multi-NVR):** L'evolució natural de la solució *single-NVR* actual consistiria a desenvolupar un mòdul de gestió centralitzada que permeti visualitzar i configurar diversos gravadors Lanaccess des d'una única instància de l'aplicació web. Mentre que la versió actual cobreix el segment d'empreses petites amb un únic dispositiu, aquesta ampliació permetria atendre grans instal·lacions corporatives amb desenes de gravadors distribuïts geogràficament.
- **Suport d'enregistrament avançat:** Ampliar les eines d'edició i exportació de vídeo directament des del navegador, permetent l'aplicació de marques d'aigua digitals o signatures legals en les descàrregues.
- **Aplicació Mòbil Nativa:** Utilitzar tecnologies com *Ionic* o *Capacitor* per reutilitzar el codi d'Angular 19 i oferir una versió nativa a les botigues d'aplicacions (App Store / Play Store).

8.3 Conclusions personals

La realització d'aquest TFG ha estat una experiència d'aprenentatge integral que ha anat molt més enllà de la simple programació.

A nivell tècnic, m'ha permès dominar les darreres funcionalitats d'Angular 19, especialment el canvi de paradigma que suposen els *Signals* per a la gestió de l'estat reactiu. Així mateix, el repte de treballar amb protocols de vídeo com WebRTC i HLS m'ha proporcionat una visió profunda sobre la gestió de fluxos multimèdia en entorns de xarxa complexos.

A nivell de gestió, el projecte m'ha ensenyat la importància de l'arquitectura *schema-driven*. Desenvolupar un software que no sàpiga exactament què ha de configurar fins que rep les dades del servidor requereix un nivell d'abstracció i rigor en el disseny que ha posat a prova els meus coneixements d'arquitectura de software i principis SOLID.

Finalment, el fet de desenvolupar el projecte per a una empresa real com Lanaccess m'ha permès entendre les necessitats de mantenibilitat i escalabilitat que exigeix el mercat industrial, on el software ha de ser, per sobre de tot, robust i eficient.

Bibliografia

- [1] C. Holmberg, H. Alvestrand i C. Jennings. *Overview: Real-Time Communication with WebRTC*. RFC 8825. IETF, 2021. URL: <https://datatracker.ietf.org/doc/html/rfc8825>.
- [2] Apple Inc. *HTTP Live Streaming (HLS) Authoring Specification*. 2023. URL: <https://developer.apple.com/documentation/http-live-streaming> (cons. 12-05-2024).
- [3] Google - Angular Team. *Angular Signals Guide*. 2024. URL: <https://angular.dev/guide/signals> (cons. 12-05-2024).
- [4] Unió Europea. *Reglament (UE) 2016/679 del Parlament Europeu i del Consell (Reglament General de Protecció de Dades - RGPD)*. DOUE-L-2016-80807. 2016. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [5] Boletín Oficial del Estado. *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD)*. BOE-A-2018-16673. 2018. URL: <https://www.boe.es/eli/es/lo/2018/12/05/3>.
- [6] Kasun Indrasiri i Danesh Kuruppu. *gRPC: Up and Running: Building High-Performance APIs with gRPC and Protocol Buffers*. O'Reilly Media, 2020. ISBN: 978-1492058335.
- [7] I. Fette i A. Melnikov. *The WebSocket Protocol*. RFC 6455. IETF, 2011. URL: <https://datatracker.ietf.org/doc/html/rfc6455>.
- [8] W3C WebRTC Working Group. *WebRTC 1.0: Real-Time Communication Between Browsers*. 2024. URL: <https://www.w3.org/TR/webrtc/> (cons. 17-05-2026).
- [9] Aristeidis Bampakos i Pablo Deeleman. *Learning Angular: Build modern web applications with the latest features of Angular*. 4th. Packt Publishing, 2023. ISBN: 978-1803233819.
- [10] M. Jones, J. Bradley i N. Sakimura. *JSON Web Token (JWT)*. RFC 7519. IETF, 2015. URL: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [11] Microsoft. *Playwright Web Testing and Automation*. 2025. URL: <https://playwright.dev> (cons. 17-05-2026).
- [12] Robert C. Martin. *The Principles of Agile Design*. 2000. URL: <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>.

- [13] gRPC Authors. *Introduction to gRPC*. 2024. URL: <https://grpc.io/docs/what-is-grpc/introduction/>.

Capítol A

Annexos

En aquest apartat s'inclouen materials addicionals que complementen la memòria del projecte.